
Particle Flow for Learning from Label Proportions

Alain Rakotomamonjy¹ Maxime Vono¹ Liva Ralaivola¹

Abstract

We introduce a novel method for *learning from label proportion*. To this end, we learn a classifier that minimizes three key objectives: (i) a bag-level loss, which quantifies the discrepancy between true and predicted label proportions in bags, (ii) an instance-level loss, inspired from domain adaptation, which leverages anchor samples with known labels and trainable supports, and (iii) a distribution discrepancy that aims at aligning anchors' learned support with those of the bag samples. Learning comes down to an alternating optimization process, iteratively updating the classifier and aligning distributions via a particle flow method. The flow of anchor samples is governed by a vector field designed to minimize the anchor loss while ensuring alignment between anchor and bag distributions. We provide a theoretical analysis, guaranteeing the convergence of the flow and identifying conditions under which the method achieves effective alignment. Our analysis highlights that separation and diversity in label proportions within bags is a critical factor for learnability. Empirical results on tabular and image datasets demonstrate the method's effectiveness, outperforming state-of-the-art approaches.

1. Introduction

Learning a classifier from labeled samples, where each sample comes with its accompanying label, is a well-understood problem (Hastie et al., 2009). However, in many real-world applications, the cost of labeling or privacy reasons prevent access to individual labels (Chang et al., 2017). Examples include trying to predict election outcomes from aggregate voting preferences, or learning bidding models in advertising from cohort-level user data due to third-party cookie deprecation on the open internet (Diemert et al., 2022). In

those situations, the task at hand may become a *weakly* supervised learning problem where training data are accessed through bags of samples only associated with some aggregate information about labels such as *label proportions*. Given such coarse sample information, the goal remains to learn a classifier able to predict the individual labels of new samples. This problem, addressed in this paper, is known as *learning from label proportions*, or LLP.

Among the different LLP paradigms, we are interested in the one in which the objective is to learn an instance-level classifier from label proportions. Within this context, Ardehaly & Culotta (2017) designed a loss function that measures the Kullback-Leibler distance between the true label proportions and the predicted label proportion of each class, estimated by aggregation. Follow-up work tackling multi-class LLP problems, by estimating instance labels or by approximating instance-level loss, include Dulac-Arnold et al. (2019); Busa-Fekete et al. (2023).

The methodology we propose here tackles this instance-level LLP problem by jointly learning from label proportions in bags and solving a distribution alignment task. We introduce this task as a way to leverage the label proportions in bags through a domain adaptation lens, in addition to the standard bag-level loss. From this domain adaptation perspective (Ben-David et al., 2010), the distribution alignment task consists in aligning the empirical distributions of the bag sample with samples with learnable supports and known labels, called here *anchor distribution*. Hence, the learning problem we solve consists in jointly minimising *i)* a bag-level loss that measures the discrepancy between true and predicted label proportions in bags, *ii)* an instance-level loss of the anchor samples with known labels and *iii)* the discrepancy between the anchors' distribution and the bags' distributions. The problem is solved through a block-coordinate descent strategy, updating the classifier and then aligning the distributions. The latter is achieved iteratively through a anchor particle flow method that minimizes the anchors loss with respect to the instance-level loss and the Wasserstein distance between the anchor distribution samples and the empirical distributions of the bags. A key aspect of our approach is that the Wasserstein distance is naturally designed to account for both the label proportions of the bags, through constraints put on marginal distributions. We provide some theoretical analysis of the anchor

¹Criteo AI Lab, Paris, France. Correspondence to: Alain Rakotomamonjy <alain.rakoto@insa-rouen.fr>.

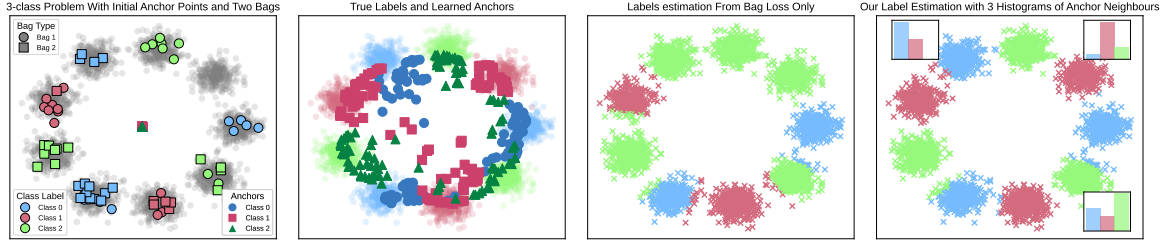


Figure 1. Our approach, FLOWLLP, learns instance-level classifier f_θ from label proportions in bags through three key phases. Left panel shows the samples in bags colored according to their true labels as well as the underlying distribution (grey). (i) Warm-up: we initialize f_θ by training only on bag-level loss. (ii) Anchor Alignment: using anchor particle flow, we optimize a set of labeled prototypes. This process jointly minimizes the Wasserstein distance to the bag distributions and the instance-level loss. Middle-left panel depicts the learned anchor samples (colored according to their labels). (iii) Refinement: we fine-tune f_θ using a mix of instance-level and bag-level losses. Impact: While training on bags alone yields inaccurate boundaries (middle-right), our anchor-driven fine-tuning produces accurate instance classifications (right). The histograms depict the label proportions of the 100 nearest anchor to the centers of those Gaussians.

particle flow and the conditions under which the proposed method achieves good alignment. Our analysis shows that one key factor for learnability is the gap and diversity in label proportions between classes in bags. Experimental results on tabular and images datasets show the effectiveness of the proposed method compared to the above-described state-of-the-art methods.

In summary, our main contributions are: *i*) a new method for learning from label proportions that leverages *domain adaptation* ideas and *particle flow*, *ii*) a theoretical analysis of the particle flow, used for distribution alignment, clarifying under which conditions the proposed method behaves correctly; *iii*) an experimental analysis confirming that gap and diversity in bags’ label proportions is a crucial factor for learning and that the proposed method, named FLOWLLP, outperforms state-of-the-art methods.

Related Work. From the perspective of domain adaptation, several works have proposed to align the distributions of source and target domains using adversarial approaches (Ganin et al., 2016), or optimal transport (Courty et al., 2017; Rakotomamonjy et al., 2022). However, our problem differs from the standard multi-domain adaptation problem in several ways (Turrise et al., 2022). First, we are not aligning the source domain with a single target domain, but with multiple target domains, the bags. Second, we are aligning a proxy source domain with known labels and learnable feature (also denoted as support) to multiple target domains with partial supervision and given features.

Literature on LLP has been flourishing over the past years due to growing user privacy concerns. Among the most recent works similar to ours, we can mention Ardehaly & Culotta (2017), Dulac-Arnold et al. (2019), and Busa-Fekete et al. (2023). Those research works have addressed the problem by reformulating a loss function tailored to the LLP problem while trying to estimate the labels of the samples in the bags. For instance, Busa-Fekete et al. (2023) proposed a

technique built on unbiased estimation of the expected loss at the instance level. Others methods (Yang et al., 2021; Nandy et al., 2022) consider self-supervised learning strategies but lack in considering instance-level supervision in the learning procedure. Very recently, Ma et al. (2025) used a combination of bag-level and instance-level losses to learn a classifier from label proportions. However, their instance-level loss is computed on pseudo-labeled samples which are specifically designed for image data. The method proposed by Liu et al. (2019), LLPGAN, is the method, applicable to any data type, that is most similar to ours. They propose to generate unlabeled samples whose distribution is similar to bag distributions using an adversarial approach, and the resulting discriminator is used as the instance-level classifier. Their discriminator is not the classical generative adversarial network (GAN) discriminator but a modified one. It classifies all classes plus a fake class and generators are trained to generate samples that resemble true samples in the discriminator feature space. Our method differs from LLPGAN in the construction of the proxy labeled distribution, which do not need an adversarial training. Hence, FLOWLLP is easier to train and can exploit pre-trained models.

Notation. From now on, we use bold notation for matrix-valued parameters in contrast to scalar ones. For $K \in \mathbb{N}^*$, we let $\Delta_K = \{\mathbf{z} \in \mathbb{R}^K : \sum_{i=1}^K z_i = 1\}$ be the probability simplex. For any natural number $n \in \mathbb{N}^*$, $[n]$ is the set of all natural numbers $\{1, \dots, n\}$.

2. Background

Wasserstein Distance. We denote by $\mathcal{P}(\mathcal{X})$ the set of probability measures in $\mathcal{X}$. Let $p \geq 1$ and define $\mathcal{P}_p(\mathcal{X})$ the subset of measures in $\mathcal{P}(\mathcal{X})$ with finite p -moment. For $\mu \in \mathcal{P}_p(\mathcal{X})$ and $\nu \in \mathcal{P}_p(\mathcal{X})$, $\Pi(\mu, \nu) \subset \mathcal{P}(\mathcal{X} \times \mathcal{X})$ is the collection of probability measures or *couplings* on $\mathcal{X} \times \mathcal{X}$. The p -Wasserstein distance $\mathcal{W}_p(\mu, \nu)$ between the measures

μ and ν is defined as

$$\mathcal{W}_p(\mu, \nu) \doteq \left(\inf_{\pi \in \Pi(\mu, \nu)} \int_{\mathcal{X} \times \mathcal{X}} d_{\mathcal{X}}^p(x, x') d\pi(x, x') \right)^{1/p}. \quad (1)$$

It is proven that the infimum in (1) is attained (Peyré et al., 2019) and any probability π which realizes the minimum is an *optimal transport plan*. In the discrete case, we denote the two marginal measures as $\mu = \sum_{i=1}^n u_i \delta_{x_i}$ and $\nu = \sum_{i=1}^m v_i \delta_{x'_i}$, with $\mathbf{u} \in \Delta_n$ and $\mathbf{v} \in \Delta_m$. The *Kantorovitch relaxation* of (1) seeks for a transportation coupling $\mathbf{P}$ that solves the problem

$$\mathcal{W}_p(\mu, \nu) \doteq \left(\min_{\mathbf{P} \in \Pi(\mathbf{u}, \mathbf{v})} \langle \mathbf{C}, \mathbf{P} \rangle \right)^{1/p}, \quad (2)$$

where $\mathbf{C} \doteq (d_{\mathcal{X}}^p(x_i, x'_j)) \in \mathbb{R}^{n \times m}$ is the matrix of all pairwise costs, and $\Pi(\mathbf{u}, \mathbf{v}) \doteq \{\mathbf{P} \in \mathbb{R}_+^{n \times m} \mid \mathbf{P}\mathbf{1} = \mathbf{u}, \mathbf{P}^\top \mathbf{1} = \mathbf{v}\}$ is the *transportation polytope* (i.e., the set of all transportation plans) between the distributions $\mathbf{u}$ and $\mathbf{v}$.

3. Bag Loss and Particle Flow

Problem Setting. We are considering the problem of learning an instance-level classifier from label proportions. Formally, given n underlying bags of labeled instances $\mathcal{B}_i = \{(\mathbf{x}_{i,j}, y_{i,j})\}_{j=1}^{n_i}$ for $i \in [n]$, our training procedure only has access to the aggregate information $\mathcal{S} = \{(\mathcal{B}_i, \mathbf{z}_i)\}_{i=1}^n$, where $\mathbf{z}_i \in \Delta_C$ is a label proportion vector of size C with $z_i^{(c)} = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbb{1}(y_{i,j} = c)$, and C is the total number of classes. We are in a weakly supervised learning setup, as only the feature vectors $\{\mathbf{x}_{i,j} \in \mathcal{X} \subset \mathbb{R}^d; j \in [n_i]\}_1^n$ are *fully* observed, but not their labels. We assume there is a joint distribution P over $\mathcal{X} \times \mathcal{Y}$ from which one can sample an observation $(\mathbf{x}, y)$ and we assume that bags' samples are drawn from P , not necessarily independently. In other words, bags may be composed of samples that are correlated such that only few classes are present in a bag. In what follows, we assume that the sampled bags are disjoint, that is two bags cannot share same samples. From $\mathcal{S}$, the goal is to learn a classifier $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, with parameter θ , that predicts the label of a new sample based on its features $\mathbf{x}$.

Learning Framework. We propose to solve this problem by taking inspiration from the domain adaptation paradigm. We jointly exploit the bag-level weak supervision at our disposal and the instance-level learning from trainable samples, that we refer to as *anchors*, with imposed labels. Anchors are learned to approximate class-conditional distributions of P , and to serve as a proxy to fully labeled data.

Assume that f_θ is the classifier with parameter θ we want to learn. With abuse of notation, when applied to a bag $\mathcal{B}_i$, f_θ computes a label prediction for each sample in the bag, and then computes the estimated label proportion. Based on this

setting, we get the following optimisation problem: where

$$\min_{\theta, \{\mathbf{a}\}} \lambda_B \sum_{i=1}^n \ell_B \left(\sum_{j \in \mathcal{B}_i} f_\theta(\mathbf{x}_{i,j}), \mathbf{z}_i \right) + \lambda \sum_k \ell(f_\theta(\mathbf{a}_k), y'_k) \quad (3)$$

$$+ \lambda_W \sum_{i=1}^n \ell_W(\mathcal{A}, \mathcal{B}_i),$$

where ℓ_B is a bag loss function (such as total variation distance or cross-entropy loss), $\ell(\cdot)$ is an instance-level loss function (such as cross-entropy loss), and ℓ_W is a bag-dependent discrepancy measure that enforces the distribution of the trainable samples $\mathcal{A} = \{(\mathbf{a}_k, y'_k)\}_{k=1}^K$ to be close to the distribution of each bag $\mathcal{B}_i$; y'_k is the predefined label of the anchor sample $\mathbf{a}_k$.

Design of ℓ_W . One of the main novelty of our approach lies in the design of the discrepancy measure ℓ_W . This discrepancy measure is going to enforce the distribution of the trainable anchor samples to be close to the underlying distribution from which the bags are drawn. Formally, the anchor samples $\mathcal{A}$ define a proxy empirical distribution μ over $\mathcal{X} \times \mathcal{Y}$. Assuming that μ is on K samples, its definition is $\mu = \sum_{k=1}^K u_k \delta_{\mathbf{a}_k, y'_k}$, $\mathbf{u} \in \Delta_K$, where $\delta_{\mathbf{a}_k, y'_k}$ is the Dirac distribution at $(\mathbf{a}_k, y'_k)$. In practice, the support $\mathbf{a}_k$ is trainable and the labels y'_k are set such that the labels of μ are equally distributed over the set of labels $\mathcal{Y}$.

Let us now focus on the design of the discrepancy measure ℓ_W . The bag $\mathcal{B}_i$ defines an empirical distribution ν_i over $\mathcal{X} \times \mathcal{Y}$ as $\nu_i = \sum_{j=1}^{n_i} v_j \delta_{\mathbf{x}_{i,j}, y_{i,j}}$. The challenge we have is that we do not have access to the labels $\{y_j\}_{j=1}^{n_i}$ of the samples in the bag. Hence, we cannot directly compute a discrepancy measure between μ and ν , but we can restrict ourselves to discrepancy measures that can be computed based on the features of the samples only. However, this would discard the information about the labels of the anchor samples as well as the label proportions of the bag. Wasserstein distances (Peyré et al., 2019) are discrepancy measures that can be computed based on the features of the samples only, while taking into account the labels of the anchor samples and the label proportion via the marginal constraints. Hence, we propose to use the Wasserstein distance $\mathcal{W}_p$, in Equation (2) as the discrepancy measure ℓ_W and we properly define the marginal constraints to take into account the labels of the anchor samples and the label proportions of the bags.

The classical choice for the marginal constraints $\mathbf{u}$ and $\mathbf{v}$, when computing the Wasserstein distance is to use the uniform weights. However, in our case, we have access to the label proportion of the bags. Hence, it is of primary importance to take this key information into account for defining the marginal constraints used to find the support of μ . In practice, we define the weight of anchors of class c so that

Algorithm 1 FlowLLP

Require: initialisation of $\{\mathbf{a}_k^{(0)}\}$ and T

- 1: Initialize classifier $f_\theta^{(0)}$ by training on bag loss only
- 2: **for** $t = 1$ **to** T **do**
- 3: Compute $\{\mathbf{a}_k^{(t)}\}$ with fixed $f_\theta^{(t-1)}$ using Algorithm 2
- 4: Train classifier $f_\theta^{(t)}$ with fixed $\{\mathbf{a}_k^{(t)}\}$
- 5: **end for**

Ensure: return f_θ^T

they are going to be proportional to the label proportion of the c -th class in the i -th bag. Hence, we choose

$$u_{k:y'_k=c}^{(i)} = \frac{z_i^{(c)} \cdot C}{K} \quad \text{and} \quad v_j = \frac{1}{n_i}. \quad (4)$$

Note that this choice of $\mathbf{u}$ enforces the mass of class c of $\sum_k u_k^{(i)} \delta_{\mathbf{a}_k, y'_k}$ to be equal to the label proportion of class c in the bag defining ν_i , hence, enforcing $z_i^{(c)} \cdot n_i$ samples in ν_i to be mapped with the K/C samples of class c in $\sum_k u_k^{(i)} \delta_{\mathbf{a}_k, y'_k}$. We will make this point clear later in the theoretical analysis section.

For the ground distance of the Wasserstein distance, we have chosen a distance that takes into account the Euclidean distance between the supports of μ and ν_i as well as a divergence $d_y(\cdot, \cdot)$ between the labels of μ and the estimated labels of ν_i . $d_{\mathcal{X} \times \mathcal{Y}}((\mathbf{a}, y'), (\mathbf{x}, f_\theta(\mathbf{x}))) = \|\mathbf{a} - \mathbf{x}\|_2 + \alpha d_y(y', f_\theta(\mathbf{x}))$, where α is a hyperparameter that controls the importance of the label divergence in the ground distance.

Algorithm. Before delving into the theoretical analysis of our approach, we present the optimisation algorithm we have designed to solve the problem in Equation (3). After initialisation of f_θ , we consider a block coordinate descent algorithm that alternates between updating the support of anchors and the classifier f_θ (see Algorithm 1). Updating the classifier parameters θ is performed by fixing the support of the anchors and computing the gradient of the bag loss and the instance-level loss using automatic differentiation.

The anchor flow for updating their support is detailed in Algorithm 2. It is performed by iterating over the bags in mini-batches of size B . For each bag in the batch, we compute the optimal transport plan $\mathbf{P}_i^*$ between the anchor distribution μ and the bag distribution ν_i . Then, we compute the gradient of the Wasserstein distance with respect to each anchor point $\mathbf{a}_k$ using the envelope theorem as $\nabla_{\mathbf{a}_k} \mathcal{W}_p(\mu, \nu_i) = \langle \nabla_{\mathbf{a}_k} \mathbf{C}, \mathbf{P}_i^* \rangle$, where $\mathbf{C}$ is the cost matrix used to compute $\mathcal{W}_p(\mu, \nu_i)$. We also compute the gradient of the instance-level loss with respect to each anchor point $\mathbf{a}_k$. Finally, we accumulate these gradients over all bags in the batch and update each anchor point $\mathbf{a}_k$ using a gradient step. This process is repeated for T_W iterations.

Algorithm 2 Anchor Particle Flow

Require: initialisation of $\{\mathbf{a}_k^{(0)}\}$, $\phi^{(0)}$, T_W , α_t , batch size B

- 1: **for** $t = 1$ **to** T_W **do**
- 2: Random sample a batch of bags $\{\mathcal{B}_i\}_{i=1}^B$
- 3: **for** $i = 1$ **to** B **do**
- 4: *// Iterate over bag in batch*
- 5: Build matrix distance using $(\mathbf{a}_k^{(t-1)}, y'_k), (\mathbf{x}_i, f_\theta(\mathbf{x}_i))$ and α_t
- 6: Build marginal constraints $\mathbf{u}^{(i)}, \mathbf{v}^{(i)}$
- 7: Compute $\mathbf{P}_i^* = \arg \min \mathcal{W}(\mu, \nu_i)$
- 8: Compute $\nabla_{\mathbf{a}_k} \mathcal{W}_p(\mu, \nu_i) = \langle \nabla_{\mathbf{a}_k} \mathbf{C}, \mathbf{P}_i^* \rangle$ for all k
- 9: Compute $\nabla_{\mathbf{a}_k} f_\theta(\mathbf{a}_k)$ for all k
- 10: Accumulate $\nabla_{\mathbf{a}_k} = \nabla_{\mathbf{a}_k} \mathcal{W}_p + \nabla_{\mathbf{a}_k} f_\theta(\mathbf{a}_k)$ for all k
- 11: **end for**
- 12: Update $\mathbf{a}_k^{(t)} = \mathbf{a}_k^{(t-1)} - \gamma_k \nabla_{\mathbf{a}_k}$ for all k
- 13: **end for**

Ensure: return $(\mathbf{a}_k^{(T_W)}, y'_k)$.

4. Theoretical Analysis of the Particle Flow

In this section, we characterize the theoretical properties of the proposed Wasserstein flow. While the full system involves a dynamic interplay between the Wasserstein flow and the classifier flow, this interaction is inherently non-convex and sensitive to specific data geometries. To establish fundamental guarantees, we first isolate the Wasserstein flow, as it serves as the primary mechanism for distributional alignment. Analyzing this component in a decoupled setting allows us to derive properties that remain invariant to the specific choice of classifier architecture and its performance. Examples of flow dynamics are illustrated in Figures 8-10 in the supplementary material.

On Its Convergence. We are especially interested in understanding under which conditions the Wasserstein-based flow is able to align class-conditional distributions of the anchor distribution to the one of the data. For that purpose, we analyze the flow as a stochastic process governed by the gradient of the Wasserstein distance on a randomly selected bag at each iteration. Details are given in the supplementary material.

Flow update. The closed-form expression of the update of each particle $\mathbf{a}_k$ of the anchor distribution, ignoring for clarity the label information and assuming ℓ_2 ground distance and $p = 2$, is

$$\mathbf{a}_{k+1} = \mathbf{a}_k + \gamma_k (\mathbf{s}_k - \mathbf{a}_k),$$

where $\mathbf{s}_k = K \sum_j \mathbf{P}_{k,j}^* \mathbf{x}_{i,j}$, is also known as the *Wasserstein barycentric mapping* of $\mathbf{a}_k$ onto the bag $\mathcal{B}_i$, $\mathbf{P}_i^*$ is the optimal coupling matrix between the anchor distribution and the bag distribution ν_i at iteration k , and γ_k is the step size at iteration k .

Given this update equation, we can now state under which conditions an anchor particle $\mathbf{a}_k$ class c is going to flow

towards the convex set C_c , the convex hull of bag samples of class c .

Theorem 1 (Almost Sure Convergence of anchor to C_c). *Assume that all samples are in the unit ball $B(0, 1)$. Let $C_c \subset B(0, 1)$ be a compact convex set. Let $\mathbf{a}_{k+1} = \mathbf{a}_k + \gamma_k(\mathbf{s}_k - \mathbf{a}_k)$ with $\|\mathbf{a}_0\| \leq 1$ and $\|\mathbf{s}_k\| \leq 1$. Let $V_k = \text{dist}(\mathbf{a}_k, C_c)^2$ and $\mathcal{H}_k$ be the history of the stochastic process up to iteration k , $\mathcal{H}_k = \{\mathbf{a}_0, \mathbf{s}_0, \dots, \mathbf{s}_{k-1}\}$. Suppose:*

1. **Error Probability:** $\mathbb{P}(\mathbf{s}_k \in C_c \mid \mathcal{H}_k) = \delta_k(V_k)$, where $(1 - \delta_k(V_k)) \leq \lambda\sqrt{V_k}$ and $\delta_k(V_k) > 2\lambda$ for some $\lambda \geq 0$.
2. **Step Sizes:** $\sum \gamma_k = \infty$ and $\sum \gamma_k^2 < \infty$.

Then $V_k \rightarrow 0$ almost surely as $k \rightarrow \infty$.

The proof of this theorem is given in Appendix A.2. One of its key aspect is the error probability condition that states the importance of $\mathbf{s}_k$ being in C_c . Note that as a barycentric mapping, $\mathbf{s}_k$ is guaranteed to be in C_c if $\mathbf{a}_k$ is coupled only to samples from its same class, that is, samples in C_c . The error probability needs to increase as the distance of $\mathbf{a}_k$ to C_c decreases. This condition, which seems to be mild in the context of optimal transport, is crucial to ensure that the stochastic process converges to C_c . In addition, we also need a minimal probability of correct coupling to ensure that the anchor particle does not get stuck far from C_c . We will see in the following analyses that this condition can be ensured by a sufficient gap in label proportions.

In the sequel, we provide two analyses stating under which conditions, we have such a matching of an anchor of class c to samples of class c in the bags. The first analysis looks at the necessary condition for the optimal transport plan to couple the class-conditional distributions of the anchor and the bags regardless of the label proportion. The second analysis exhibits the key role of the label proportion in the matching problem.

In order to simplify the problem, we are going to analyze this matching problem at the class-conditional distribution level. We denote as μ_c the class-conditional distribution of the anchor distribution and ν_c the class-conditional distribution of a given bag and $d(\cdot, \cdot)$ a distance between distributions.

Guarantee on class-conditional matching. We want to flow particles of class-conditional distributions to their corresponding true class-conditional distributions. So, the question that we answer here is which conditions are needed to ensure correct matching of class-conditional distributions between the anchor and the bags?

Proposition 1. *A necessary condition for the optimal plan $\mathbf{P}^*$ between $\mu = \sum_{c=1}^C a_c \mu_c$ and $\nu = \sum_{c=1}^C a_c \nu_c$ to couple μ_c to ν_c for all c is that the class-conditionals are*

are geometrically arranged such that, for any permutation σ of the classes, we have, we have, $\sum_{c=1}^C d(\mu_c, \nu_c) \leq \sum_{c=1}^C d(\mu_c, \nu_{\sigma(c)})$, when $d(\cdot, \cdot)$ is the Wasserstein distance.

The proof of this proposition is based on direct applications of results by Santambrogio (2015). Continuity of the Wasserstein distance is given in Proposition 7.4 of Santambrogio (2015) and the proof of the proposition follows from Theorem 1.38 of the same book.

While this condition may be circonvoluted, there exists simpler and more intuitive conditions that can be derived from it. For instance, a situation where the class-conditional distributions are so that $d(\mu_c, \nu_c) < d(\mu_c, \nu_{c'})$ for all c and c' satisfies this necessary condition and provably guarantee a correct matching of class-conditionals. The intuition behind this condition is that, when marginal constraints are matched as defined in Equation (4), there exists a geometrical condition that ensures correct matching of class-conditional distributions for the Wasserstein optimal transport plan. Next analysis also shows that this condition can be relaxed when we take into account the label proportions in the bags.

Gap in label proportion. We now analyze how knowing label proportion and adjusting anchor class-conditional weights can be a key factor for ensuring correct matching. Due to the combinatorial nature of the multiclass problem, we restrict ourself here to binary classification problem, and we also assume that the label proportions are a_0 and a_1 with $a_1 > a_0$, without loss of generality. In such context, the distribution ν in a bag is going to be a mixture of the class-conditional distributions $\nu = a_0 \nu_0 + a_1 \nu_1$. Owing to the reweighting label proportion strategy we propose, the anchor distribution is going to be a mixture of the class-conditional $\mu = a_0 \mu_0 + a_1 \mu_1$ and this reweighting scheme ensures us the following property (proof in Appendix A.3):

Proposition 2. *Under the above assumptions, the Wasserstein optimal transport plan between μ and ν is going to match at least a proportion $a_1 - a_0$ of mass of distribution between same class μ_c and ν_c for all c class-conditional distributions.*

Based on this proposition, we can see that the gap in label proportion is going to play a crucial role in the problem. Indeed, $a_1 - a_0$ mass of anchor distribution (of the majority class) is going to be coupled with samples of the same class in the bag, making the anchor flow relevant. We understand from this proposition that we need bags with large gap in label proportion and the class of largest proportion need to vary across the bags. This proposition ensures that a substantial part of the anchor distributions will be correctly matched under the Wasserstein optimal transport plan.

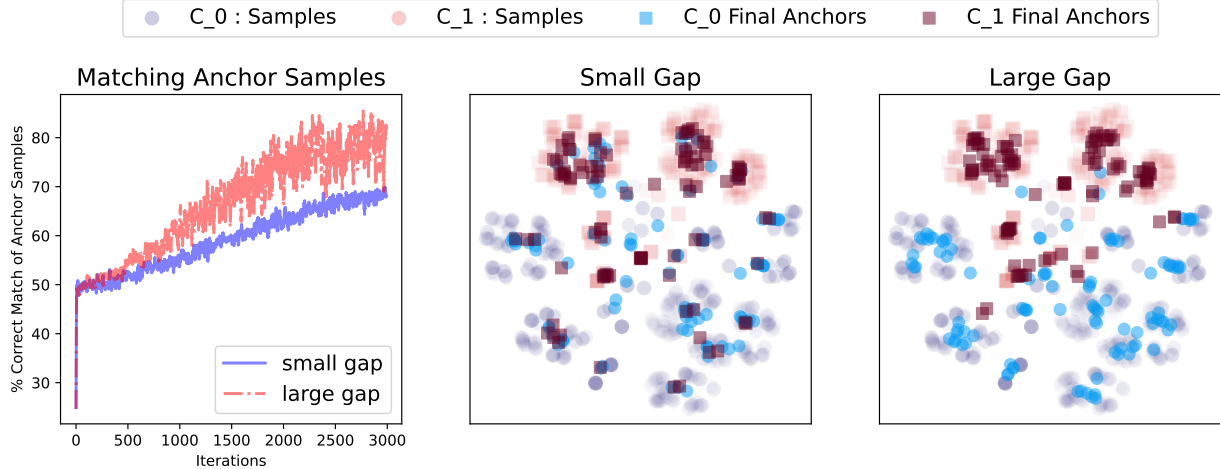


Figure 2. Impact of the label proportion on the anchor matching. (left) We show the temporally-averaged ratio of correct matching of anchor particles to samples of same class in the bags when having small and large gap in label proportions for the binary “Mushrooms” classification problem. We also show the anchors in these two situations based on their 2D TSNE representation after training. (middle) small gap. (right) large gap. on the middle plot, we can see that some anchors have not converge to the support of their correct class either C_0 or C_1 .

Table 1. Summary of performance of the different algorithms on multiclass (MC), 54 different dataset/bag size experiments and binary (B) datasets, 18 different dataset/bag size experiments. Excerpt and full tables are respectively available in Table 2 and in the supplementary material. (Table 6 and Table 7). Best performances are in bold. The number in parentheses indicates the number of experiments where the method achieves the best performance.

Data	Meta Dataset	
	MC - 54	Binary - 18
Full	90.8 $\pm$ 0 (0)	88.3 $\pm$ 1 (0)
DAOT	40.3 $\pm$ 6 (0)	62.2 $\pm$ 5 (0)
Easy LLP	55.1 $\pm$ 6 (0)	75.7 $\pm$ 5 (0)
Entropy loss	88.1 $\pm$ 1 (1)	83.3 $\pm$ 1 (1)
LLPGAN	88.4 $\pm$ 1 (9)	83.6 $\pm$ 1 (7)
FlowLLP	89.1 $\pm$ 1 (44)	83.7 $\pm$ 1 (10)

5. Numerical Experiments

LLP methods. We compare the performance of FlowLLP, with several baselines, that aim at predicting instance labels and designed to handle multiclass learning problem. The first baseline, denoted as Full is a model trained on samples and labels of all instances. This baseline delivers an upper bound on performance for the problem. The second baseline, denoted as DA (Dulac-Arnold et al., 2019), learns a model jointly from a bag-level loss and an instance-level loss by estimating individual labels in each bag. Labels are enforced to match the bag label proportion. The baseline EasyLLP algorithm learns an instance-level model by training on the bag-level labels through a debiased loss. LLPGAN is the generative adversarial network-based LLP method proposed by Liu et al. (2019). It uses a genera-

tor to generate instances conditioned on the bag-level labels and a discriminator to classify instances. In addition to the GAN framework, LLPGAN uses a cross-entropy based loss to enforce consistency between bag-level true and predicted label proportion. The baseline Entropy Loss is the method that uses only this loss. For all the LLP algorithms, we have used the same instance-level neural network model architecture. The code of FlowLLP is available at <https://github.com/arakotom/flowllp>.

Bag construction. We consider a bag sampling that generates diversity in label proportions accross bags. They are constructed as follows. For each bag, we first sample the number of classes in the bag, then sample the classes to be present in the bag and finally sample a random number of instances from the selected classes. By doing so, we ensure that bags comply with a certain level of diversity in class distribution and in label proportion. We made vary the number of classes in bags from 2 to C where C is the number of classes in the dataset.

Datasets and Evaluation metrics. We use binary and multiclass tabular datasets and an image dataset CIFAR10. Characteristics of the datasets are summarized in Table 4. Details on data pre-processing are given in the appendix. The tabular datasets are split into training and test sets, while for the image dataset, we use the natural test set. For the former families of datasets, respectively 30% and 10% of the data are respectively used as a test and validation set. The training bags are constructed from the remaining data. The LLP algorithms are trained on those bags. The performance of the algorithms is evaluated on the test set using the

Table 2. Excerpt from the table results (see Table 6) on multiclass datasets. Best performances are in bold and second best are underlined.

Data	Algo	Number of bags								
		50			100			250		
		#class in bag			#class in bag			#class in bag		
		2	C-1	C	2	C-1	C	2	C-1	C
Dry Beans	Full	94.07 $\pm$ 0.2	94.16 $\pm$ 0.3	94.21 $\pm$ 0.3	94.16 $\pm$ 0.3	94.13 $\pm$ 0.4	94.21 $\pm$ 0.3	94.12 $\pm$ 0.3	94.01 $\pm$ 0.3	94.19 $\pm$ 0.2
	DAOT	87.79 $\pm$ 5.9	23.96 $\pm$ 4.8	24.52 $\pm$ 3.8	84.57 $\pm$ 8.5	27.61 $\pm$ 4.5	23.45 $\pm$ 5.3	75.58 $\pm$ 10.6	30.21 $\pm$ 4.1	26.84 $\pm$ 6.2
	Easy LLP	71.04 $\pm$ 8.8	69.41 $\pm$ 5.6	67.04 $\pm$ 8.9	57.41 $\pm$ 7.2	61.56 $\pm$ 10.7	61.12 $\pm$ 8.7	45.01 $\pm$ 6.5	44.73 $\pm$ 6.7	49.49 $\pm$ 5.7
	Entropy loss	93.46 $\pm$ 0.5	93.29 $\pm$ 0.3	93.30 $\pm$ 0.4	93.33 $\pm$ 0.4	93.15 $\pm$ 0.3	93.15 $\pm$ 0.3	90.40 $\pm$ 6.0	93.22 $\pm$ 0.4	93.06 $\pm$ 0.3
	LLPGAN	93.81 $\pm$ 0.3	93.62 $\pm$ 0.4	93.57 $\pm$ 0.3	93.49 $\pm$ 0.4	93.53 $\pm$ 0.2	93.34 $\pm$ 0.4	91.62 $\pm$ 4.4	93.28 $\pm$ 0.4	93.21 $\pm$ 0.3
	FlowLLP	93.69 $\pm$ 0.2	93.48 $\pm$ 0.4	93.52 $\pm$ 0.4	93.63 $\pm$ 0.3	93.60 $\pm$ 0.3	93.38 $\pm$ 0.4	93.73 $\pm$ 0.3	93.45 $\pm$ 0.2	93.28 $\pm$ 0.3
HAR	Full	98.16 $\pm$ 0.3	98.17 $\pm$ 0.2	98.08 $\pm$ 0.3	98.14 $\pm$ 0.2	98.12 $\pm$ 0.2	98.22 $\pm$ 0.2	98.14 $\pm$ 0.3	98.17 $\pm$ 0.3	98.18 $\pm$ 0.3
	DAOT	72.43 $\pm$ 12.4	40.49 $\pm$ 3.6	37.91 $\pm$ 4.0	62.10 $\pm$ 12.4	40.95 $\pm$ 4.3	37.14 $\pm$ 3.8	60.55 $\pm$ 12.2	37.53 $\pm$ 3.3	34.99 $\pm$ 1.9
	Easy LLP	68.97 $\pm$ 9.6	81.79 $\pm$ 9.2	84.11 $\pm$ 7.2	59.17 $\pm$ 10.0	67.33 $\pm$ 6.7	67.17 $\pm$ 9.1	46.39 $\pm$ 3.3	45.53 $\pm$ 7.4	46.58 $\pm$ 6.9
	Entropy loss	97.15 $\pm$ 0.4	96.42 $\pm$ 0.3	96.34 $\pm$ 0.4	96.77 $\pm$ 0.4	96.22 $\pm$ 0.3	95.64 $\pm$ 0.5	96.62 $\pm$ 0.3	95.37 $\pm$ 0.7	94.18 $\pm$ 0.7
	LLPGAN	96.95 $\pm$ 0.4	96.48 $\pm$ 0.5	96.00 $\pm$ 0.5	96.75 $\pm$ 0.5	96.36 $\pm$ 0.3	95.71 $\pm$ 0.3	96.35 $\pm$ 0.4	95.76 $\pm$ 0.5	94.78 $\pm$ 0.4
	FlowLLP	97.68 $\pm$ 0.4	97.05 $\pm$ 0.5	96.95 $\pm$ 0.4	97.79 $\pm$ 0.3	97.06 $\pm$ 0.5	96.82 $\pm$ 0.4	97.87 $\pm$ 0.2	96.84 $\pm$ 0.3	96.15 $\pm$ 0.4
CIFAR	Full	86.40 $\pm$ 0.2	86.41 $\pm$ 0.2	86.45 $\pm$ 0.2	86.31 $\pm$ 0.2	86.48 $\pm$ 0.2	86.50 $\pm$ 0.3	86.48 $\pm$ 0.2	86.40 $\pm$ 0.2	86.39 $\pm$ 0.3
	DAOT	77.65 $\pm$ 1.1	16.71 $\pm$ 3.1	14.40 $\pm$ 3.3	76.43 $\pm$ 2.8	15.18 $\pm$ 2.5	16.47 $\pm$ 2.1	76.02 $\pm$ 4.4	18.86 $\pm$ 4.4	16.57 $\pm$ 3.6
	Easy LLP	49.10 $\pm$ 3.9	76.96 $\pm$ 2.0	77.39 $\pm$ 2.0	37.84 $\pm$ 5.5	62.19 $\pm$ 3.7	62.03 $\pm$ 4.6	27.43 $\pm$ 6.1	32.19 $\pm$ 7.0	36.71 $\pm$ 5.4
	Entropy loss	85.35 $\pm$ 0.2	84.22 $\pm$ 0.2	84.13 $\pm$ 0.2	85.23 $\pm$ 0.1	83.89 $\pm$ 0.2	83.89 $\pm$ 0.2	84.86 $\pm$ 0.1	83.76 $\pm$ 0.2	83.68 $\pm$ 0.2
	LLPGAN	85.43 $\pm$ 0.1	84.53 $\pm$ 0.2	84.43 $\pm$ 0.3	85.60 $\pm$ 0.3	84.44 $\pm$ 0.2	84.33 $\pm$ 0.3	85.72 $\pm$ 0.3	84.56 $\pm$ 0.2	84.20 $\pm$ 0.4
	FlowLLP	85.59 $\pm$ 0.2	84.65 $\pm$ 0.2	84.63 $\pm$ 0.2	85.63 $\pm$ 0.2	84.47 $\pm$ 0.3	84.62 $\pm$ 0.2	85.77 $\pm$ 0.1	<u>84.32 $\pm$ 0.2</u>	84.47 $\pm$ 0.2

balanced accuracy metric. For each dataset and bag settings, the same model is trained using the different algorithms and the results are averaged over ten runs (random split of the data and bags).

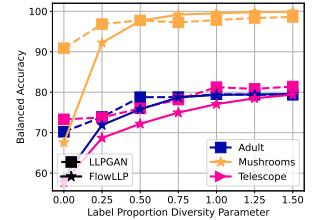
Specific settings for FlowLLP. For our model, we have chosen the following setting that has been used for all different datasets and bag settings. We have set only one iteration ($T = 1$) of block-coordinate descent described in Algorithm 1. The number of anchor samples per class is set to $K = 2000$ and the number of flow steps has been set to $T_W = 3000$ with a bag batch size B set to 1. The learning rate is set to 10^{-4} . λ_B has been set to 1 while λ and λ_W share the same value and this value has been validated to balance the trade-off between the bag-level loss and the anchor loss. For the ground distance, α which weights distance on $\mathcal{X}$ and $\mathcal{Y}$ has been set to 10. These parameter settings have been used for all the experiments.

Results. Results of all the experiments are summarized in Table 1. While Table 2 presents an excerpt of the results for the multiclass datasets, full detailed results are reported in the appendix, in Table 6 and Table 7. Our findings can be summarized as follows. For multiclass datasets, FlowLLP outperforms the other LLP algorithms on most of the datasets and bag settings (see Table 1). It achieves the best performance on 44 out of 54 experiments. For binary datasets, FlowLLP also outperforms the other LLP algorithms but with a smaller margin (see Table 1) and Table 7. We believe this is due to the fact that multiclass problems generates more diverse label proportions in the bags, which is a key ingredient for the success of FlowLLP. Note that the benefit of the diversity in the label proportion in bags is a well known theoretical statement (Scott & Zhang, 2019) that FlowLLP plainly exploits and that we have confirmed

empirically.

Anchor matching. One of the key success factor of FlowLLP is the weakly-supervised anchor flow based on optimal transport. Our theoretical analysis shows that the benefit of this anchor flow relies on the correct matching of anchor samples to same-class samples in the bags. Besides, it is also make clear that the larger the gap in the label proportion, the better the matching accuracy. We have conducted experiments to confirm this theoretical finding. From the “Mushrooms” (easy) binary classification problem, we have constructed bags of size 50. We separate the bags in two sets of bags for which label proportion gaps are respectively smaller and larger than 0.6. We then run FlowLLP on those two sets of bags and monitor the correct matching of the anchor samples at each computation of the optimal transport plan. The results are depicted in Figure 2. We can see that the matching accuracy grows rapidly higher when training with bags of larger gap in label proportion. The benefit of these correct matching is confirmed by the evolution of the anchor samples during the training where, when the gap is small, some anchors do not converge to the support of their correct class.

Effect of gap and diversity. We have conducted further experiments to evaluate the impact of gap and diversity on FlowLLP and LLPGAN. As such, we considered a controlled setting for building bags in binary tabular datasets. For each bag, we fixed the number n_m of samples from the mi-


 Figure 3. Impact of the label proportion diversity parameter δ .

nority class and then set the number of samples from the majority class as $n_M = (\beta + u) \cdot n_m$ where β and u are parameters respectively that control the gap in label proportion and the diversity, as u is randomly sampled from a uniform distribution in $[-\frac{\delta}{2}, \frac{\delta}{2}]$. We have run two sets of experiments, one where $u = 0$ and β is varied and the other where $\beta = 1$ and u is varied. The results are summarized in Figure 3 and Figure 15. They show that diversity in label proportion has a strong significant impact on the performance of both algorithms and that the larger the diversity, the better the performance for both algorithms. and when there is no diversity ($u = 0$), LLPGAN should be preferred to FlowLLP, even when the gap is large (see Figure 15).

End-to-end training. FlowLLP can be easily adapted to an end-to-end learning with the strong benefit of leveraging pre-trained models. Key adaptations are as follows: (i) feature extraction: a pre-trained ResNet18 (ImageNet) is used as a feature extractor producing a 512-dimensional latent space. (ii) dimensionality reduction: a one-layer neural network reduces the latent space to 50 dimensions, addressing the curse of dimensionality for Wasserstein distance. (iii) anchor flow: anchor samples are defined in this latent 50-dimensional space and are optimized via particle flow as in FlowLLP. (iv) classification: a fully connected neural network outputs class probabilities from the sample embeddings. From the training side, since anchors are defined in the latent space, with anchor loss affecting only the classifier, while bag-level loss impacts the entire model, we adapt the training as follows in three steps: (1) at initialization, we freeze the feature extractor and train the dimensionality reduction layer and classifier using the bag loss. (2) then we learn the anchor distribution via particle flow optimization. (3) and we fine-tune all components (except anchors) using bag loss and anchor-based loss. We repeat steps (2) and (3) until the number of iterations T is reached.

Results on CIFAR10 with bags of size 50 and different image sizes are reported in Figure 4 (more experiments are reported in the appendix, including multiple iteration, $T > 1$). It shows the benefit of leveraging pre-trained models in LLP, as well as the strong gain in performance brought by FlowLLP compared to using a pre-trained model without our anchor-based approach and a bag-level loss only. We can also note that the weight λ of the anchor loss has an impact on the performance as well as on the variance across runs. We do not report comparison with LLPGAN as we were not able to make it work in this end-to-end setting despite our efforts (see Remark in Appendix A.6 for details). This also stresses the simplicity of FlowLLP to be adapted to different settings.

Above results have been obtained with a bottleneck of 50 dimensions for the anchors. The primary motivation was to

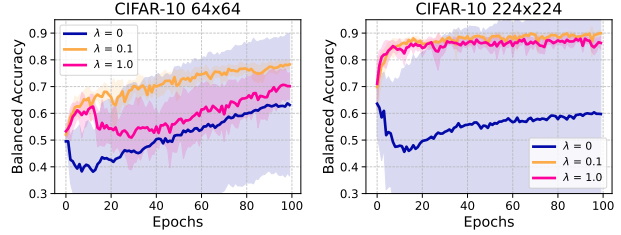


Figure 4. End-to-end training on CIFAR10 with bags of size 50. We show the evolution of accuracy across epochs during the fine-tuning of f_θ once anchors have been learned. Performance is shown for different values of λ (weight of the anchor loss).

avoid computing Optimal Transport in the raw pixel space, where standard metrics are often uninformative and this bottleneck strategy brings dimensionality reduction that is crucial for tempering curse of dimensionality in Wasserstein distance. Nonetheless, we have also conducted an ablation study on this latent dimension in the context of 64×64 image size. We have considered the following dimensions for the bottleneck: 50, 128, 256 and 512 (the latter being the original dimension of the ResNet feature layer, hence the dimensionality reduction function being the identity function). The performance of the algorithm for these different dimensions are respectively 79.6%, 80.9%, 81.2% and 83.4%. These results show that the performance of the algorithm is better when the bottleneck dimension is larger, with the best performance obtained without dimensionality reduction. This suggests that, in this end-to-end setting, the curse of dimensionality is not a strong issue for computing Wasserstein distance and that the bottleneck strategy may not be necessary. However, it is important to note that this may not be the case for all datasets and settings, and that further experiments are needed to draw more general conclusions on this point.

Runtime analysis. The use of optimal transport and Wasserstein distance in FlowLLP raises the question of the computational cost of the algorithm. While the theoretical complexity of exact optimal transport is cubic with respect to the number of samples per bag, it is important to note that for most practical LLP applications, bag sizes remain within a range where this computational overhead is tractable. In addition, there exists Sinkhorn-based or low-rank-based alternative solvers that can be employed to further reduce this complexity if needed.

To grasp insights regarding scalability, we conduct a runtime analysis on end-to-end learning CIFAR10 using a ResNet-18 architecture on a V100 GPU, using the same default settings as above. The results, summarized in the table below, track the duration (in seconds) of each phase of our alternating algorithm across varying bag sizes.

These results in Table 3 demonstrate that the computational

Table 3. Runtime analysis across varying bag sizes on end-to-end training on CIFAR10 and on a single GPU (NVIDIA V100). We report the time taken for each phase of the algorithm: bag loss training, anchor flow (including particle flow and OT computation), classifier training, and the total time. All times are in seconds.

Bag Size	Bag loss	Anchor	Classifier	Total
25	1400	900	900	3200
50	1200	1400	1000	3600
100	1100	2100	1300	4500
200	1100	3300	2900	6300

cost of the anchor training (which includes the particle flow and OT computation) scales gracefully. Even when quadrupling the bag size from 50 to 200, the total runtime is less than the double. This confirms that the overhead of our method remains comparable to the final training phase, ensuring its practical usability for complex datasets.

For tabular datasets, we can also observe (Figure 16) that the running time of `FlowLLP` is higher than the one of `LLPGAN`. However, for end-to-end training on images, we conjecture that `FlowLLP` is computationally more efficient than `LLPGAN` due to the sensitivity of GAN training. Hence, beyond running time, the practical usability of our approach is also evidenced by its ability to easily handle different types of datasets (tabular, images with size 224x224) easily, compared to `LLPGAN` which is highly unstable on images.

Further analyses. We have conducted further analyses to understand the behavior of the algorithm with the following conclusions (details in the appendix).

- The number of anchor samples slightly impacts the performance of `FlowLLP`. However, a trade-off may need to be found with respect to the running time (Figure 12).
- The label regularization in the ground cost of the Wasserstein distance has a small impact on the performance of `FlowLLP` (Figure 14).
- The initialization of the anchor distribution has a small impact on the performance (Figure 13). We found that the larger the variance of the initialization, the better the performance. We conjecture that this is due to the classifier flow that helps correcting a poor initialization.
- Under noisy label proportions, `FlowLLP` is more sensitive to noise than `LLPGAN` especially for large noise levels (Figure 11).

6. Conclusion

We have introduced `FlowLLP`, a novel method for learning from label proportions. By reformulating LLP as a distribution alignment problem, our method employs a learnable anchor distribution and particle flow to bridge the gap be-

tween bag-level information and individual sample labels.

We have conducted a theoretical analysis of the particle flow process, in order to better understand its dynamics and identify the essential conditions required for accurate label estimation. Our findings highlight that a critical factor influencing learnability is the gap and diversity in label proportions among classes within the bags.

Empirical evaluations on both tabular and image datasets demonstrate the effectiveness of `FlowLLP` in comparison to existing approaches. The ablation studies confirm the critical role of bag diversity in label proportions for successful learning.

Impact Statement

This work has significant implications for real-world applications where only aggregated label statistics are available, such as privacy-preserving learning, medical diagnosis, and computational advertising. By bridging the gap between bag-level and instance-level learning, this research paves the way for more robust and scalable solutions in weakly supervised learning scenarios. One risk associated with this research is the potential misuse of the proposed method for leaking sensitive information from the data. To mitigate this risk, practitioners should ensure that the method is only applied to data where the privacy of individuals is protected.

Acknowledgments

This work was granted access to the HPC / AI resources of IDRIS under the allocation 2024- A0171015707 and 2025- A0191015707 made by GENCI (Grand Equipement National de Calcul Intensif).

References

- Ardehaly, E. M. and Culotta, A. Co-training for demographic classification using deep learning from label proportions. In *2017 IEEE International Conference on Data Mining Workshops (ICDMW)*, pp. 1017–1024. Ieee, 2017.
- Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F., and Vaughan, J. W. A theory of learning from different domains. *Machine learning*, 79:151–175, 2010.
- Busa-Fekete, R., Choi, H., Dick, T., Gentile, C., and Munoz Medina, A. Easy learning from label proportions. *Advances in Neural Information Processing Systems*, 36: 14957–14968, 2023.
- Chang, J. C., Amershi, S., and Kamar, E. Revolt: Collaborative crowdsourcing for labeling machine learning datasets. In *Proceedings of the 2017 CHI Confer-*

- ence on Human Factors in Computing Systems, CHI '17, pp. 2334–2346, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450346559. doi: 10.1145/3025453.3026044. URL <https://doi.org/10.1145/3025453.3026044>.
- Courty, N., Flamary, R., Tuia, D., and Rakotomamonjy, A. Optimal transport for domain adaptation. *IEEE transactions on pattern analysis and machine intelligence*, 39(9): 1853–1865, 2017.
- Diemert, E., Fabre, R., Gilotte, A., Jia, F., Leparmentier, B., Mary, J., Qu, Z., Tanielian, U., and Yang, H. Lessons from the adkdd’21 privacy-preserving ml challenge. In *Proceedings of the ACM Web Conference 2022*, WWW ’22, pp. 2026–2035, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450390965. doi: 10.1145/3485447.3512076. URL <https://doi.org/10.1145/3485447.3512076>.
- Dulac-Arnold, G., Zeghidour, N., Cuturi, M., Beyer, L., and Vert, J.-P. Deep multi-class learning from label proportions. *arXiv preprint arXiv:1905.12909*, 2019.
- Fan, K., Zhang, H., Yan, S., Wang, L., Zhang, W., and Feng, J. Learning a generative classifier from label proportions. *Neurocomputing*, 139:47–55, 2014.
- Flamary, R., Courty, N., Gramfort, A., Alaya, M. Z., Boissunon, A., Chambon, S., Chapel, L., Corenflos, A., Fatras, K., Fournier, N., Gautheron, L., Gayraud, N. T., Janati, H., Rakotomamonjy, A., Redko, I., Rolet, A., Schutz, A., Seguy, V., Sutherland, D. J., Tavenard, R., Tong, A., and Vayer, T. Pot: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021. URL <http://jmlr.org/papers/v22/20-451.html>.
- Franceschi, J.-Y., Gartrell, M., Dos Santos, L., Issenhuth, T., de Bézenac, E., Chen, M., and Rakotomamonjy, A. Unifying gans and score-based diffusion as generative particle models. *Advances in Neural Information Processing Systems*, 36:59729–59760, 2023.
- Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., and Lempitsky, V. Domain-adversarial training of neural networks. *The Journal of Machine Learning Research*, 17(1):2096–2030, 2016.
- Hastie, T., Tibshirani, R., and Friedman, J. *The elements of statistical learning: data mining, inference and prediction*. Springer, 2 edition, 2009.
- Havaladar, S., Sharma, N., Sareen, S., Shanmugam, K., and Raghuveer, A. Learning from label proportions: Bootstrapping supervised learners via belief propagation. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=KQe9tHd0k8>.
- Liu, J., Wang, B., Qi, Z., Tian, Y., and Shi, Y. Learning from label proportions with generative adversarial networks. *Advances in neural information processing systems*, 32, 2019.
- Ma, T., Chen, H., Hu, J., Zhu, Y., and Li, X. Forming auxiliary high-confident instance-level loss to promote learning from label proportions. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 20592–20601, 2025.
- Musicant, D. R., Christensen, J. M., and Olson, J. F. Supervised learning by training on aggregate outputs. In *Seventh IEEE International Conference on Data Mining (ICDM 2007)*, pp. 252–261. IEEE, 2007.
- Nandy, J., Saket, R., Jain, P., Chauhan, J., Ravindran, B., and Raghuveer, A. Domain-agnostic contrastive representations for learning from label proportions. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pp. 1542–1551, 2022.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. *ArXiv*, abs/1912.01703, 2019. URL <https://api.semanticscholar.org/CorpusID:202786778>.
- Patrini, G., Nock, R., Rivera, P., and Caetano, T. (almost) no label no cry. *Advances in Neural Information Processing Systems*, 27, 2014.
- Peyré, G., Cuturi, M., et al. Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5-6):355–607, 2019.
- Quadrianto, N., Smola, A. J., Caetano, T. S., and Le, Q. V. Estimating labels from label proportions. In *Proceedings of the 25th international conference on Machine learning*, pp. 776–783, 2008.
- Rakotomamonjy, A., Flamary, R., Gasso, G., Alaya, M. E., Berar, M., and Courty, N. Optimal transport for conditional domain matching and label shift. *Machine Learning*, pp. 1–20, 2022.
- Robbins, H. and Siegmund, D. A convergence theorem for non negative almost supermartingales and some applications. In *Optimizing methods in statistics*, pp. 233–257. Elsevier, 1971.

- Santambrogio, F. Optimal transport for applied mathematicians. *Birkäuser, NY*, 55(58-63):94, 2015.
- Scott, C. and Zhang, J. Learning from multiple corrupted sources, with application to learning from label proportions. *arXiv preprint arXiv:1910.04665*, 2019.
- Turrisi, R., Flamary, R., Rakotomamonjy, A., and Pontil, M. Multi-source domain adaptation via weighted joint distributions optimal transport. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2022.
- Yang, H., Zhang, W., and Lam, W. A two-stage training framework with feature-label matching mechanism for learning from label proportions. In *Asian Conference on Machine Learning*, pp. 1461–1476. PMLR, 2021.
- Yu, F., Liu, D., Kumar, S., Tony, J., and Chang, S.-F. α -SVM for learning with label proportions. In Dasgupta, S. and McAllester, D. (eds.), *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pp. 504–512, Atlanta, Georgia, USA, 17–19 Jun 2013. PMLR. URL <https://proceedings.mlr.press/v28/yl13a.html>.

A. Appendix

A.1. Additional literature on Learning from Label Proportions.

We must mention a parallel LLP line of work, that, given a bag/set of test instances, focuses on predicting the proportions of labels in this bag/set, discarding the question of predicting labels instance by instance. [Musicant et al. \(2007\)](#); [Yu et al. \(2013\)](#) proposed a method to train a classical supervised learning algorithm through a modification of the instance-level loss function in neural networks to accomodate aggregate outputs of hidden layers. Albeit providing satisfying results in specific scenarios ([Quadrianto et al., 2008](#); [Fan et al., 2014](#); [Patrini et al., 2014](#)), learning a good bag-level estimator does not guarantee, in the general case, that individual labels will be correctly estimated ([Scott & Zhang, 2019](#)).

A.2. Convergence of anchor to its class-conditionals.

Assumption on data. Let us consider that all samples in the dataset are contained in are of norm less than 1, i.e., $\|\mathbf{x}_{i,j}\|_2 \leq 1$ for all $i \in [n]$ and $j \in [n_i]$. We define as C_c the compact convex hull of all samples of class c in the dataset, i.e., $C_c = \text{Conv}(\{\mathbf{x}_{i,j} : y_{i,j} = c, i \in [n], j \in [n_i]\})$. We assume for simplicity that each convex hull C_c is built from a single mode class-conditional distribution.

Update rule for anchor sample. Let us consider an anchor sample $\mathbf{a}_k$ of class c . When using ℓ_2 norm as cost function in the Wasserstein distance, without label-based distance, the Wasserstein update rule for a given iteration and for a single bag is given by:

$$\mathbf{a}_{k+1} = \mathbf{a}_k - \gamma'_k \nabla_{\mathbf{a}_k} \mathcal{W}_2(\mu, \nu_i), \quad (5)$$

$$= \mathbf{a}_k - 2\gamma'_k \sum_j^{n_i} \mathbf{P}_{k,j}^* (\mathbf{a}_k - \mathbf{x}_{i,j}), \quad (6)$$

$$= \mathbf{a}_k - 2 \frac{\gamma'_k}{K} (\mathbf{a}_k - \mathbf{s}_k), \quad (7)$$

$$= \mathbf{a}_k + \gamma_k (\mathbf{s}_k - \mathbf{a}_k), \quad (8)$$

where $\mathbf{s}_k = K \sum_j^{n_i} \mathbf{P}_{k,j}^* \mathbf{x}_{i,j}$ is the barycentric mapping of $\mathbf{a}_k$ in the support of the bag ν_i , and $\gamma_k = \frac{2\gamma'_k}{K}$ is the step size at iteration k . Note that since $\mathbf{x}_{i,j}$ are of norm less than 1, then $\|\mathbf{s}_k\|_2 \leq 1$. Note that the gradient is obtained owing to the envelope theorem.

Now, let us consider the following theorem that states the conditions under which an anchor sample converges almost surely to the convex set C_c of its empirical class-conditional. For this purpose, we will consider the anchor update rule as a stochastic process where at each iteration, the barycentric mapping $\mathbf{s}_k$ is a random variable dependent on the bag sampled at iteration k . We monitor the distance between the anchor sample and the convex set C_c through the variable $V_k = \text{dist}(\mathbf{a}_k, C_c)^2 \triangleq \min_{\mathbf{x} \in C_c} \|\mathbf{a}_k - \mathbf{x}\|^2$. One of the key assumptions is that the barycentric mapping $\mathbf{s}_k$ falls inside the convex set C_c with a probability that increases as the anchor sample $\mathbf{a}_k$ gets closer to C_c . This is a very natural assumption in an optimal transport setting as the anchor sample will be coupled with samples from its own class more likely and more frequently as it gets closer to those samples.

Theorem 2 (Almost Sure Convergence of anchor to C_c). *Let $C_c \subset B(0, 1)$ be a compact convex set. Let $\mathbf{a}_{k+1} = \mathbf{a}_k + \gamma_k (\mathbf{s}_k - \mathbf{a}_k)$ with $\|\mathbf{a}_0\| \leq 1$ and $\|\mathbf{s}_k\| \leq 1$. Let $V_k = \text{dist}(\mathbf{a}_k, C_c)^2$ and $\mathcal{H}_k$ be the history of the stochastic process up to iteration k $\mathcal{H}_k = \{\mathbf{a}_0, \mathbf{s}_0, \dots, \mathbf{s}_{k-1}\}$. Suppose:*

1. **Error Probability:** $\mathbb{P}(\mathbf{s}_k \in C_c \mid \mathcal{H}_k) = \delta_k(V_k)$, where $(1 - \delta_k(V_k)) \leq \lambda \sqrt{V_k}$ and $\delta_k(V_k) > 2\lambda$ for some $\lambda \geq 0$.

2. **Step Sizes:** $\sum \gamma_k = \infty$ and $\sum \gamma_k^2 < \infty$.

Then $V_k \rightarrow 0$ almost surely as $k \rightarrow \infty$.

Proof.

Geometric Stability. First, we show that the stochastic process $\{\mathbf{a}_k\}$ is stable in the sense that $\|\mathbf{a}_k\| \leq 1$ for all k . This implies that the maximum possible distance between any two points in the walk is 2. Since the unit-ball $B(0, 1)$ is convex and $\mathbf{a}_0, \mathbf{s}_k \in B(0, 1)$, the update $\mathbf{a}_{k+1} = (1 - \gamma_k)\mathbf{a}_k + \gamma_k \mathbf{s}_k$ is a convex combination. Thus, $\|\mathbf{a}_k\| \leq 1$ for all k . This

implies the maximum possible distance between any two points in the walk is 2.

Bounding the Distance to the Convex Set. Let $\mathbf{p}_k \triangleq \Pi_C(\mathbf{a}_k) = \arg \min_{\mathbf{x} \in C} \|\mathbf{a}_k - \mathbf{x}\|$. Since $\mathbf{p}_k \in C$, we have $V_{k+1} \leq \|\mathbf{a}_{k+1} - \mathbf{p}_k\|^2$. Expanding the update:

$$V_{k+1} \leq \|(\mathbf{a}_k - \mathbf{p}_k) + \gamma_k(\mathbf{s}_k - \mathbf{a}_k)\|^2 \quad (9)$$

$$= \|\mathbf{a}_k - \mathbf{p}_k\|^2 + 2\gamma_k \langle \mathbf{a}_k - \mathbf{p}_k, \mathbf{s}_k - \mathbf{a}_k \rangle + \gamma_k^2 \|\mathbf{s}_k - \mathbf{a}_k\|^2 \quad (10)$$

Conditional Expectation. We are interested in the expected behavior of V_{k+1} with respect to the randomness of $\mathbf{s}_k$, conditioned on the history $\mathcal{H}_k$. So, taking the expectation conditioned on $\mathcal{H}_k$:

$$\mathbb{E}[V_{k+1} \mid \mathcal{H}_k] \leq V_k + 2\gamma_k \mathbb{E}[\langle \mathbf{a}_k - \mathbf{p}_k, \mathbf{s}_k - \mathbf{a}_k \rangle \mid \mathcal{H}_k] + 4\gamma_k^2 \quad (11)$$

Note that V_k and $\mathbf{a}_k$ are fixed given $\mathcal{H}_k$ and $\|\mathbf{s}_k - \mathbf{a}_k\|^2 \leq (\|\mathbf{s}_k\| + \|\mathbf{a}_k\|)^2 \leq 2^2 = 4$ because they are both in the unit ball. This inequality is reminiscent to inequality used in the analysis of stochastic processes where the second term is usually called the drift term.

Drift Term Decomposition. We now focus on bounding the drift term depending on whether $\mathbf{s}_k$ falls inside the convex set C_c or not. Let $A_k = \{\mathbf{s}_k \in C\}$ and $\delta_k = P(A_k \mid \mathcal{H}_k)$. Using the law of total expectation, we split the inner product:

$$\mathbb{E}[\langle \dots \rangle \mid \mathcal{H}_k] = \delta_k \mathbb{E}[\langle \mathbf{a}_k - \mathbf{p}_k, \mathbf{s}_k - \mathbf{a}_k \rangle \mid A_k] + (1 - \delta_k) \mathbb{E}[\langle \mathbf{a}_k - \mathbf{p}_k, \mathbf{s}_k - \mathbf{a}_k \rangle \mid A_k^c]$$

- **Case $\mathbf{s}_k \in C$:** By the projection property, $\langle \mathbf{a}_k - \mathbf{p}_k, \mathbf{s}_k - \mathbf{a}_k \rangle \leq -V_k$.
- **Case $\mathbf{s}_k \notin C$:** By Cauchy-Schwarz, $\langle \mathbf{a}_k - \mathbf{p}_k, \mathbf{s}_k - \mathbf{a}_k \rangle \leq \|\mathbf{a}_k - \mathbf{p}_k\| \cdot \|\mathbf{s}_k - \mathbf{a}_k\| \leq 2\sqrt{V_k}$.

Substitute these into the drift term and applying the vanishing matching Error assumption: Substitute $(1 - \delta_k) \leq \lambda\sqrt{V_k}$:

$$\mathbb{E}[\langle \dots \rangle \mid \mathcal{H}_k] \leq -\delta_k V_k + 2(1 - \delta_k)\sqrt{V_k} \quad (12)$$

$$\leq -\delta_k V_k + 2(\lambda\sqrt{V_k})\sqrt{V_k} = -V_k(\delta_k - 2\lambda) \quad (13)$$

Plugging this back into the main inequality Equation (11) gives:

$$\mathbb{E}[V_{k+1} \mid \mathcal{H}_k] \leq V_k - 2\gamma_k(\delta_k - 2\lambda)V_k + 4\gamma_k^2 \quad (14)$$

Applying Robbins-Siegmund theorem In order to show that V_k converges to 0 almost surely, we apply the Robbins-Siegmund theorem (Robbins & Siegmund, 1971) to Equation (14)

We have the form $\mathbb{E}[V_{k+1} \mid \mathcal{H}_k] \leq V_k - b_k + c_k$:

- $c_k = 4\gamma_k^2$: This is summable ($\sum c_k < \infty$) by assumption.
- $b_k = 2\gamma_k(\delta_k - 2\lambda)V_k$: Since by assumption, we have $\delta_k - 2\lambda > 0$, then $b_k \geq 0$.

Hence, the Robbins-Siegmund theorem guarantees that V_k converges to a finite limit V_∞ and $\sum b_k < \infty$ almost surely. We now show that $V_\infty = 0$ almost surely by contradiction. For this purpose, we are going to analyse the behavior of $\sum b_k$.

From our derivation, the drift term is:

$$b_k = 2\gamma_k(\delta_k - 2\lambda)V_k$$

We want to show that there exists a neighborhood of V_k around 0 where the drift term $(\delta_k - 2\lambda)$ is strictly positive. Intuitively, this means that when the anchor sample is close enough to the convex set C_c , the probability of the barycentric mapping $\mathbf{s}_k$ falling inside C_c is high enough to ensure a consistent “pull” towards C_c .

More formally, we assumed that $\delta(V) \rightarrow 1$ as $V \rightarrow 0$ and $2\lambda < 1$, hence, there exists a positive gap $1 - 2\lambda$ between the limit of $\delta(V)$ and 2λ . If we arbitrarily consider $\eta = \frac{1-2\lambda}{2}$. By the definition of a limit, there exists a neighborhood radius $\epsilon > 0$ such that:

$$\text{For all } V \in (0, \epsilon], \quad \delta(V) > 2\lambda + \eta$$

In this neighborhood, the drift coefficient $(\delta_k - 2\lambda)$ is guaranteed to be at least $\eta > 0$. Intuitively, this means that once the anchor sample is close enough to C_c by a distance less than ϵ , there is a guaranteed "pull" towards C_c .

Now let's prove by contradiction that $V_\infty = 0$ almost surely. Suppose $V_\infty > 0$. This would mean the stochastic process eventually "stalls" at some positive distance from the set. If this were true, then for all sufficiently large k (say $k \geq K$), we would have V_k stays above a threshold, e.g., $V_k \geq \frac{V_\infty}{2}$. In addition, since $(\delta_k - 2\lambda) > 0$, there exists a constant $c > 0$ such that $(\delta_k - 2\lambda) \geq c > 0$. Now let us look at the tail of the sum $\sum b_k$:

$$\sum_{k=K}^{\infty} b_k = \sum_{k=K}^{\infty} 2\gamma_k(\delta_k - 2\lambda)V_k \geq \sum_{k=K}^{\infty} 2\gamma_k \left(c \cdot \frac{V_\infty}{2} \right) = (c \cdot V_\infty) \sum_{k=K}^{\infty} \gamma_k$$

Because $\sum \gamma_k = \infty$, the right-hand side is infinite. This directly contradicts the Robbins-Siegmund requirement that the total sum of drift must be finite. Hence V_∞ can not be positive almost surely. Hence, the distance $V_k = 0$ almost surely, meaning $\mathbf{a}_k \rightarrow C_c$. $\square$

Remark 1. The key assumption of the theorem is the the probability of the barycentric mapping $\mathbf{s}_k$ falling inside the convex set C_c . The necessity for a minimum probability of falling inside C_c (i.e., $\delta_k > 2\lambda$) stems from the need to ensure a consistent "pull" of the anchor sample towards C_c anywhere in the space. If this probability were too low, the anchor sample could drift away from C_c indefinitely. In addition, the condition that $(1 - \delta_k) \leq \lambda\sqrt{V_k}$ ensures that as the anchor sample approaches C_c (i.e., as $V_k \rightarrow 0$), the likelihood of the barycentric mapping being outside C_c diminishes sufficiently fast. For these two situations, the impact of optimal transport on δ_k is crucial as the coupling between the anchor sample and the bag samples depends on their relative positions and on marginal constraints. Intuitively, as the anchor sample gets closer to its class-conditional distribution, it is more likely to be coupled with samples from that class, increasing δ_k . In addition, when there exists a large gap between label proportions of different classes in the bags, the majority of samples in a bag will belong to a single class. This increases the likelihood that the barycentric mapping of an anchor sample from that class will fall inside the convex set of that class, thus increasing δ_k , even when the anchor sample is not close to the convex set yet.

A.3. Proposition 2 and proof

In order to ease the analysis of matching problem, we reformulate the problem as an optimal transport problem in the space of probability measures, endowed with a metric like the Wasserstein distance. For a binary classification problem, we consider that the measure ν of the bag's class-conditional distributions is a weighted sum of Dirac masses at the location of the class-conditionals. Hence, $\nu = a_0\delta_{\nu_0} + a_1\delta_{\nu_1}$ where δ_{ν_i} is the Dirac mass at the location of the class-conditional distribution ν_i and a_i is the weight of the Dirac mass. The indices 0 and 1 refer to the two class respectively. By construction, the anchor distribution measure μ is also a weighted sum of Dirac masses at the location of the class-conditional (that needs to be learn). And again, by construction, the weight of the Dirac masses are the same as the one of the bag's class-conditional distributions. Hence, $\mu = b_0\delta_{\mu_0} + b_1\delta_{\mu_1}$.

The optimal transport problem is then to find the optimal coupling between the two measures ν and μ . The optimal coupling is a joint distribution π on the product space of the two measures that minimizes the cost of transporting the mass from ν to μ . The cost of transporting a unit of mass from ν_i to μ_j is the distance between the two class-conditionals. The optimal coupling is then the solution of the following optimization problem:

$$\pi^* = \arg \min_{\pi \in \Pi(\nu, \mu)} \sum_{i,j} \mathcal{W}(\nu_i, \mu_j) \pi_{ij} \quad (15)$$

We have two possible situations for the optimal coupling. The first one is when $\mathcal{W}(\nu_0, \mu_0) \leq \mathcal{W}(\nu_0, \mu_1)$ and $\mathcal{W}(\nu_1, \mu_1) \leq \mathcal{W}(\nu_1, \mu_0)$. In this case, the optimal coupling is $\pi^* = \begin{pmatrix} a_0 & 0 \\ 0 & a_1 \end{pmatrix}$. The second situation is when $\mathcal{W}(\nu_0, \mu_0) \geq \mathcal{W}(\nu_0, \mu_1)$ and $\mathcal{W}(\nu_1, \mu_1) \geq \mathcal{W}(\nu_1, \mu_0)$. In this case, the optimal coupling is $\pi^* = \begin{pmatrix} 0 & a_0 \\ a_0 & a_1 - a_0 \end{pmatrix}$.

This shows that the optimal coupling ensures that at least a mass $a_1 - a_0$ of is transported from two class-conditionals of same class.

A.4. Discussion on similarity and dissimilarity between LLPGAN and FlowLLP.

- Both methods generate samples to match the bags’ distributions. Our method uses a particle flow approach while LLPGAN uses a GAN approach. However, our flow using only the discriminator has the same flavor as GAN training and bear resemblance with the discriminator flow (Franceschi et al., 2023).
- Our approach explicitly uses label proportions in the distribution matching task while LLPGAN does not (in their adversarial loss).
- FlowLLP is simpler than LLPGAN to train as there is no adversarial training. In addition, the adversarial training of LLPGAN can be unstable and requires careful tuning. Indeed, it differs from classical GAN in the sense that the discriminator is not a binary classifier and the generator needs to generate samples that are similar to the real samples in the feature space of the discriminator.
- Our method can take advantage of pre-trained models.
- When bags lacks of diversity in label proportions, LLPGAN might have an advantage in terms of performance.

Table 4. UCI and OpenML dataset summary.

Dataset	# Examples	# Features	# Class	min size class	max size class
adult	48842	14	2	11687	37155
dry beans	13611	16	7	522	3546
har	10299	561	6	1406	1944
indoor	21048	528	5	1141	5464
mushroom	8124	112	2	3916	4208
musk	6598	166	2	1017	5581
optdigits	5620	64	10	554	572
phoneme	5404	5	2	1586	3818
students	4424	36	3	794	2209
telescope	19020	10	2	6688	12332
wine	4898	11	2	1060	3838

Table 5. URLs of the datasets.

Dataset	URL
adult	https://archive.ics.uci.edu/dataset/2/adult
dry beans	https://archive.ics.uci.edu/dataset/602/dry+bean+dataset
har	https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones
indoor	https://archive.ics.uci.edu/dataset/310/ujiindoorloc
mushroom	https://archive.ics.uci.edu/dataset/73/mushroom
musk	https://archive.ics.uci.edu/dataset/75/musk+version+2
optdigits	https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits
phoneme	https://www.openml.org/d/31
students	https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success
telescope	https://www.openml.org/search?type=data&sort=runs&id=1120&status=active
wine	https://archive.ics.uci.edu/dataset/186/wine+quality

A.5. Details on experimental setting

A.5.1. DATASET SUMMARY

We have used several tabular datasets from the UCI repository and OpenML for our experiments. The datasets are summarized in Table 4 in term of characteristic and they are available at the following URLs in Table 5. In our experiments, for each run, these tabular datasets are randomly split into a training, validation and test set with a ratio of 60%, 10% and 30% respectively. At this point, data are standardized using the mean and standard deviation of the training set and the validation and test sets are standardized using the same mean and standard deviation.

For the computer vision MNIST and CIFAR datasets. we have used the usual train-test split and have split the training

Table 6. results on Multiclass datasets in an class-based dependent samplings of bags. Best performances are in bold and second best are underlined.

Data	Algo	Number of bags								
		50			100			1250		
		#class in bag			#class in bag			#class in bag		
		2	C-1	C	2	C-1	C	2	c-1	c
Dry Beans	Full	94.07 $\pm$ 0.2	94.16 $\pm$ 0.3	94.21 $\pm$ 0.3	94.16 $\pm$ 0.3	94.13 $\pm$ 0.4	94.21 $\pm$ 0.3	94.12 $\pm$ 0.3	94.01 $\pm$ 0.3	94.19 $\pm$ 0.2
	DAOT	87.79 $\pm$ 5.9	23.96 $\pm$ 4.8	24.52 $\pm$ 3.8	84.57 $\pm$ 8.5	27.61 $\pm$ 4.5	23.45 $\pm$ 5.3	75.58 $\pm$ 10.6	30.21 $\pm$ 4.1	26.84 $\pm$ 6.2
	Easy LLP	71.04 $\pm$ 8.8	69.41 $\pm$ 5.6	67.04 $\pm$ 8.9	57.41 $\pm$ 7.2	61.56 $\pm$ 10.7	61.12 $\pm$ 8.7	45.01 $\pm$ 6.5	44.73 $\pm$ 6.7	49.49 $\pm$ 5.7
	Entropy loss	93.46 $\pm$ 0.5	93.29 $\pm$ 0.3	93.30 $\pm$ 0.4	93.33 $\pm$ 0.4	93.15 $\pm$ 0.3	93.15 $\pm$ 0.3	90.40 $\pm$ 6.0	93.22 $\pm$ 0.4	93.06 $\pm$ 0.3
	LLPGAN	93.81 $\pm$ 0.3	93.62 $\pm$ 0.4	93.57 $\pm$ 0.3	93.49 $\pm$ 0.4	93.53 $\pm$ 0.2	93.34 $\pm$ 0.4	91.62 $\pm$ 4.4	93.28 $\pm$ 0.4	93.21 $\pm$ 0.3
HAR	FlowLLP	93.69 $\pm$ 0.2	93.48 $\pm$ 0.4	93.52 $\pm$ 0.4	93.63 $\pm$ 0.3	93.60 $\pm$ 0.3	93.38 $\pm$ 0.4	93.73 $\pm$ 0.3	93.45 $\pm$ 0.2	93.28 $\pm$ 0.3
	Full	98.16 $\pm$ 0.3	98.17 $\pm$ 0.2	98.08 $\pm$ 0.3	98.14 $\pm$ 0.2	98.12 $\pm$ 0.2	98.22 $\pm$ 0.2	98.14 $\pm$ 0.3	98.17 $\pm$ 0.3	98.18 $\pm$ 0.3
	DAOT	72.43 $\pm$ 12.4	40.49 $\pm$ 3.6	37.91 $\pm$ 4.0	62.10 $\pm$ 12.4	40.95 $\pm$ 4.3	37.14 $\pm$ 3.8	60.55 $\pm$ 12.2	37.53 $\pm$ 3.3	34.99 $\pm$ 1.9
	Easy LLP	68.97 $\pm$ 9.6	81.79 $\pm$ 9.2	84.11 $\pm$ 7.2	59.17 $\pm$ 10.0	67.33 $\pm$ 6.7	67.17 $\pm$ 9.1	46.39 $\pm$ 3.3	45.53 $\pm$ 7.4	46.58 $\pm$ 6.9
	Entropy loss	97.15 $\pm$ 0.4	96.42 $\pm$ 0.3	96.34 $\pm$ 0.4	96.77 $\pm$ 0.4	96.22 $\pm$ 0.3	95.64 $\pm$ 0.5	96.62 $\pm$ 0.3	95.37 $\pm$ 0.7	94.18 $\pm$ 0.7
Indoor	LLPGAN	96.95 $\pm$ 0.4	96.48 $\pm$ 0.5	96.00 $\pm$ 0.5	96.75 $\pm$ 0.5	96.36 $\pm$ 0.3	95.71 $\pm$ 0.3	96.35 $\pm$ 0.4	95.76 $\pm$ 0.5	94.78 $\pm$ 0.4
	FlowLLP	97.68 $\pm$ 0.4	97.05 $\pm$ 0.5	96.95 $\pm$ 0.4	97.79 $\pm$ 0.3	97.06 $\pm$ 0.5	96.82 $\pm$ 0.4	97.87 $\pm$ 0.2	96.84 $\pm$ 0.3	96.15 $\pm$ 0.4
	Full	97.11 $\pm$ 0.2	97.08 $\pm$ 0.2	97.13 $\pm$ 0.2	97.13 $\pm$ 0.3	97.14 $\pm$ 0.3	97.20 $\pm$ 0.3	97.17 $\pm$ 0.3	97.23 $\pm$ 0.3	97.15 $\pm$ 0.3
	DAOT	56.28 $\pm$ 14.4	23.34 $\pm$ 2.6	23.25 $\pm$ 1.9	56.40 $\pm$ 13.5	25.36 $\pm$ 2.8	24.94 $\pm$ 2.9	49.93 $\pm$ 12.6	25.22 $\pm$ 3.5	25.58 $\pm$ 1.8
	Easy LLP	47.55 $\pm$ 2.9	49.40 $\pm$ 3.9	52.21 $\pm$ 3.5	39.46 $\pm$ 3.9	41.78 $\pm$ 4.0	41.12 $\pm$ 4.8	30.46 $\pm$ 4.3	34.29 $\pm$ 4.3	32.56 $\pm$ 3.7
Optdigits	Entropy loss	94.72 $\pm$ 0.3	93.40 $\pm$ 0.4	92.55 $\pm$ 0.5	94.31 $\pm$ 0.5	92.99 $\pm$ 0.8	91.78 $\pm$ 0.5	94.02 $\pm$ 0.4	92.36 $\pm$ 0.5	90.85 $\pm$ 0.7
	LLPGAN	94.75 $\pm$ 0.4	93.35 $\pm$ 0.6	91.68 $\pm$ 1.0	94.56 $\pm$ 0.6	92.76 $\pm$ 0.6	90.47 $\pm$ 0.7	94.72 $\pm$ 0.5	92.03 $\pm$ 0.6	89.84 $\pm$ 0.7
	FlowLLP	96.15 $\pm$ 0.4	94.67 $\pm$ 0.6	94.03 $\pm$ 0.4	96.25 $\pm$ 0.6	95.21 $\pm$ 0.7	92.87 $\pm$ 0.5	96.55 $\pm$ 0.4	95.01 $\pm$ 0.7	92.96 $\pm$ 0.7
	Full	98.00 $\pm$ 0.2	98.12 $\pm$ 0.2	98.13 $\pm$ 0.2	98.17 $\pm$ 0.1	98.04 $\pm$ 0.2	98.17 $\pm$ 0.2	98.08 $\pm$ 0.2	98.19 $\pm$ 0.2	98.18 $\pm$ 0.2
	DAOT	68.78 $\pm$ 14.5	21.11 $\pm$ 6.2	18.16 $\pm$ 4.7	52.50 $\pm$ 20.6	20.01 $\pm$ 4.4	18.55 $\pm$ 3.6	33.64 $\pm$ 10.7	18.82 $\pm$ 4.2	16.71 $\pm$ 5.5
Students	Easy LLP	83.11 $\pm$ 8.6	79.60 $\pm$ 4.3	75.99 $\pm$ 9.2	57.94 $\pm$ 9.0	70.33 $\pm$ 12.0	55.01 $\pm$ 11.4	41.41 $\pm$ 7.6	41.75 $\pm$ 10.3	36.51 $\pm$ 7.2
	Entropy loss	97.27 $\pm$ 0.3	96.43 $\pm$ 0.5	96.66 $\pm$ 0.5	97.27 $\pm$ 0.5	95.70 $\pm$ 0.5	95.62 $\pm$ 0.7	96.99 $\pm$ 0.4	94.67 $\pm$ 0.5	93.70 $\pm$ 1.0
	LLPGAN	97.78 $\pm$ 0.4	98.03 $\pm$ 0.4	97.82 $\pm$ 0.3	97.37 $\pm$ 0.6	97.13 $\pm$ 0.3	96.96 $\pm$ 0.7	96.68 $\pm$ 0.6	95.06 $\pm$ 0.9	94.83 $\pm$ 1.0
	FlowLLP	98.62 $\pm$ 0.2	98.11 $\pm$ 0.3	97.96 $\pm$ 0.2	98.49 $\pm$ 0.3	97.74 $\pm$ 0.3	97.33 $\pm$ 0.5	98.49 $\pm$ 0.2	96.98 $\pm$ 0.4	96.05 $\pm$ 0.6
	Full	71.07 $\pm$ 1.2	71.07 $\pm$ 1.2	70.70 $\pm$ 1.1	70.57 $\pm$ 1.4	70.57 $\pm$ 1.4	70.45 $\pm$ 1.5	70.81 $\pm$ 0.8	70.81 $\pm$ 0.8	71.05 $\pm$ 0.9
CIFAR	DAOT	55.94 $\pm$ 1.9	55.94 $\pm$ 1.9	41.01 $\pm$ 5.1	50.64 $\pm$ 6.9	50.64 $\pm$ 6.9	47.08 $\pm$ 7.0	50.63 $\pm$ 9.5	50.63 $\pm$ 9.5	40.78 $\pm$ 6.9
	Easy LLP	59.19 $\pm$ 2.5	59.19 $\pm$ 2.5	57.65 $\pm$ 1.1	57.80 $\pm$ 1.5	57.80 $\pm$ 1.5	56.36 $\pm$ 1.2	53.04 $\pm$ 6.7	53.04 $\pm$ 6.7	53.23 $\pm$ 6.8
	Entropy loss	66.84 $\pm$ 1.5	66.84 $\pm$ 1.5	65.56 $\pm$ 1.7	67.52 $\pm$ 0.8	67.52 $\pm$ 0.8	65.77 $\pm$ 1.4	66.43 $\pm$ 1.6	66.43 $\pm$ 1.6	65.05 $\pm$ 1.2
	LLPGAN	67.54 $\pm$ 1.3	67.54 $\pm$ 1.3	64.68 $\pm$ 2.4	67.39 $\pm$ 1.0	67.39 $\pm$ 1.0	65.76 $\pm$ 1.3	67.25 $\pm$ 1.4	67.25 $\pm$ 1.4	65.23 $\pm$ 0.9
	FlowLLP	67.50 $\pm$ 1.5	67.50 $\pm$ 1.5	65.83 $\pm$ 2.2	67.57 $\pm$ 1.0	67.57 $\pm$ 1.0	64.47 $\pm$ 1.7	66.89 $\pm$ 1.5	66.89 $\pm$ 1.5	64.58 $\pm$ 1.3
CIFAR	Full	86.40 $\pm$ 0.2	86.41 $\pm$ 0.2	86.45 $\pm$ 0.2	86.31 $\pm$ 0.2	86.48 $\pm$ 0.2	86.50 $\pm$ 0.3	86.48 $\pm$ 0.2	86.40 $\pm$ 0.2	86.39 $\pm$ 0.3
	DAOT	77.65 $\pm$ 1.1	16.71 $\pm$ 3.1	14.40 $\pm$ 3.3	76.43 $\pm$ 2.8	15.18 $\pm$ 2.5	16.47 $\pm$ 2.1	76.02 $\pm$ 4.4	18.86 $\pm$ 4.4	16.57 $\pm$ 3.6
	Easy LLP	49.10 $\pm$ 3.9	76.96 $\pm$ 2.0	77.39 $\pm$ 2.0	37.84 $\pm$ 5.5	62.19 $\pm$ 3.7	62.03 $\pm$ 4.6	27.43 $\pm$ 6.1	32.19 $\pm$ 7.0	36.71 $\pm$ 5.4
	Entropy loss	85.35 $\pm$ 0.2	84.22 $\pm$ 0.2	84.13 $\pm$ 0.2	85.23 $\pm$ 0.1	83.89 $\pm$ 0.2	83.89 $\pm$ 0.2	84.86 $\pm$ 0.1	83.76 $\pm$ 0.2	83.68 $\pm$ 0.2
	LLPGAN	85.43 $\pm$ 0.1	84.53 $\pm$ 0.2	84.43 $\pm$ 0.3	85.60 $\pm$ 0.3	84.44 $\pm$ 0.2	84.33 $\pm$ 0.3	85.72 $\pm$ 0.3	84.56 $\pm$ 0.2	84.20 $\pm$ 0.4
CIFAR	FlowLLP	85.59 $\pm$ 0.2	84.65 $\pm$ 0.2	84.63 $\pm$ 0.2	85.63 $\pm$ 0.2	84.47 $\pm$ 0.3	84.62 $\pm$ 0.2	85.77 $\pm$ 0.1	<u>84.32 $\pm$ 0.2</u>	84.47 $\pm$ 0.2

set into training and validation sets with a ratio of 90% and 10% respectively. They have been normalized in a standard way channel-wise. When not in the end-to-end training, for the CIFAR dataset, as a common practice in the LLP literature (Havaladar et al., 2024), we have considered a latent space and have embedded the samples in a 50-dimensional space using a pre-trained ResNet18 model and a PCA projection. Note that for the bag size we have considered (50, 100, and 250 samples), reported multiclass results on CIFAR peak around 50% of accuracy for the DA (Dulac-Arnold et al., 2019) while using the proposed embedding, their model and our implementation reaches around 77% of accuracy. So all models benefit from the embedding.

A.5.2. BAG CONSTRUCTION IN THE EXPERIMENTS

As described in the main part of the paper, we have considered a stratified sampling of the bags. For each bag, given a pre-defined number of class in the bag, we sample the class present in the bag. Given the number of times a class is present in all the bag, we randomly split the samples of each class to be assigned to bags and then assign samples to bags. Note that for this sampling strategy, since bag size can vary, the training data is split based on a number of bag instead of bag size. However, we have set the number of bags for each dataset to be the ratio of training samples and bag size so that the mean bag size is approximately a pre-defined bag size. For all the experiments, we have considered 10 runs, and bag sizes of 50, 100, and 250 samples. For the multi-class datasets with C classes, we have considered a situation where only 2, $C - 1$ and all C classes are present in the bags. This allows us to evaluate the impact of the number of classes in the bags on the performance of the algorithms.

A.5.3. MODEL AND TRAINING

For the tabular datasets, we have considered a fully connected neural network with three hidden layers with 500,200 and 200 units respectively and a bias. The activation function is a Leaky ReLU with a 0.2 parameter.

Table 7. Performance on binary datasets with an class-based dependent samplings of bags.

Data	Algo	Number of bags		
		50	100	250
Adult	Full	82.19 $\pm$ 0.2	82.20 $\pm$ 0.3	82.18 $\pm$ 0.2
	DAOT	50.86 $\pm$ 1.5	52.48 $\pm$ 2.4	53.71 $\pm$ 2.1
	Easy LLP	77.09 $\pm$ 0.8	76.51 $\pm$ 1.2	68.55 $\pm$ 5.7
	Entropy loss	79.85 $\pm$ 0.3	79.49 $\pm$ 0.6	79.64 $\pm$ 0.5
	LLPGAN	80.07 $\pm$ 0.4	79.83 $\pm$ 0.6	79.65 $\pm$ 0.5
	FlowLLP	79.79 $\pm$ 0.2	79.89 $\pm$ 0.5	79.46 $\pm$ 0.7
Mushrooms	Full	99.97 $\pm$ 0.0	100.00 $\pm$ 0.0	100.00 $\pm$ 0.0
	DAOT	89.89 $\pm$ 2.1	89.83 $\pm$ 2.0	87.06 $\pm$ 10.0
	Easy LLP	98.92 $\pm$ 0.7	98.54 $\pm$ 0.7	94.31 $\pm$ 3.4
	Entropy loss	99.94 $\pm$ 0.1	99.91 $\pm$ 0.1	99.76 $\pm$ 0.3
	LLPGAN	99.66 $\pm$ 0.4	99.33 $\pm$ 0.5	99.18 $\pm$ 0.5
	FlowLLP	99.98 $\pm$ 0.0	99.96 $\pm$ 0.0	99.84 $\pm$ 0.2
Musk	Full	97.70 $\pm$ 0.5	97.92 $\pm$ 0.7	97.64 $\pm$ 0.5
	DAOT	56.16 $\pm$ 4.7	55.87 $\pm$ 6.0	57.11 $\pm$ 5.8
	Easy LLP	77.52 $\pm$ 3.8	75.45 $\pm$ 2.9	68.27 $\pm$ 7.5
	Entropy loss	87.75 $\pm$ 1.8	88.10 $\pm$ 2.3	86.81 $\pm$ 1.4
	LLPGAN	89.12 $\pm$ 2.0	88.52 $\pm$ 2.9	85.38 $\pm$ 2.3
	FlowLLP	88.43 $\pm$ 2.3	88.87 $\pm$ 2.6	86.86 $\pm$ 2.8
Phoneme	Full	85.59 $\pm$ 0.9	85.59 $\pm$ 1.2	86.16 $\pm$ 0.9
	DAOT	56.14 $\pm$ 6.2	62.88 $\pm$ 9.3	61.03 $\pm$ 7.2
	Easy LLP	75.10 $\pm$ 3.3	72.20 $\pm$ 8.3	66.65 $\pm$ 11.5
	Entropy loss	78.34 $\pm$ 1.5	77.73 $\pm$ 1.3	77.92 $\pm$ 1.6
	LLPGAN	78.55 $\pm$ 1.4	78.43 $\pm$ 2.0	77.82 $\pm$ 1.2
	FlowLLP	80.25 $\pm$ 1.5	79.48 $\pm$ 1.0	80.18 $\pm$ 2.0
Telescope	Full	85.60 $\pm$ 0.6	85.86 $\pm$ 0.5	85.50 $\pm$ 0.6
	DAOT	57.07 $\pm$ 3.7	58.31 $\pm$ 5.5	59.36 $\pm$ 2.9
	Easy LLP	78.28 $\pm$ 0.8	75.59 $\pm$ 1.3	70.12 $\pm$ 8.0
	Entropy loss	83.05 $\pm$ 0.5	83.00 $\pm$ 0.6	82.77 $\pm$ 1.0
	LLPGAN	85.54 $\pm$ 0.3	84.88 $\pm$ 0.5	83.90 $\pm$ 0.5
	FlowLLP	83.36 $\pm$ 0.5	83.23 $\pm$ 0.4	83.36 $\pm$ 0.4
Wine	Full	78.05 $\pm$ 1.1	78.52 $\pm$ 1.5	78.43 $\pm$ 1.3
	DAOT	55.14 $\pm$ 4.3	58.62 $\pm$ 3.6	58.91 $\pm$ 6.7
	Easy LLP	65.77 $\pm$ 6.3	61.05 $\pm$ 9.4	62.51 $\pm$ 7.9
	Entropy loss	71.41 $\pm$ 1.3	72.15 $\pm$ 1.4	72.28 $\pm$ 1.4
	LLPGAN	71.02 $\pm$ 1.8	72.20 $\pm$ 1.6	72.15 $\pm$ 2.0
	FlowLLP	71.43 $\pm$ 1.4	71.85 $\pm$ 1.7	71.22 $\pm$ 2.9

- EasyLLP is trained for 1000 epochs with an SGD optimizer with a learning rate defined as an hyperparameter and a momentum of 0.9. Tested learning rates are 0.01, 0.001, 0.0001 and 0.00001. We picked the best learning rate based on the validation set.
- Similarly, DA is trained for 1000 epochs with an SGD optimizer with a learning rate defined as an hyperparameter, a momentum of 0.9 and a weight decay of 10^{-4} . Tested learning rates are 0.01, 0.001, 0.0001 and 0.00001. We picked the best learning rate based on the validation set.
- We have reimplemented LLPGAN based on the details given in Liu et al. (2019) and the publicly available code¹ written in Tensorflow 1.x. The generator is a fully connected neural networks with three hidden layers with 512, 1024 and 2048 units respectively and a bias. The discriminator is also fully connected neural networks with three hidden layers with 2048, 1024 and 512 units respectively and a bias. Unlike classical GANs, the output layer of the discriminator is a softmax layer with C outputs where C is the number of classes. The input of the generator is a Gaussian noise vector of dimension 64. The activation function is a Leaky ReLU. LLPGAN is trained for 100 epochs using Adam optimizers for generator and discriminator with one update of the generator per update of the discriminator for each bag. The learning rate of the optimizers as well as the regularization weight between the (entropy) bag loss and the adversarial loss are set as hyperparameters. The learning rate is validated among 0.01, 0.001, 0.0001 and 0.00001 while the regularization weight is validated among [0.1, 0.5, 1, 5, 10] (authors recommend to set it to 1).
- For the Entropy minimization baseline, we have used the same codebase as for LLPGAN and removed the generator and adversarial part of the code. Training is similarly done for 100 epochs.
- for FlowLLP, the loss function is the cross-entropy loss for instance-level classification of the learned anchor samples and the bag-level loss is a cross-entropy loss between the predicted label proportions in the bags and the provided label proportions, as for LLPGAN and Entropy minimization. The classifier architecture is the same as for the other methods. The anchor distribution is initialized as a mixture of Gaussian distributions if zero-mean and isotropic covariance $\sigma^2 I$ (with $\sigma = 2$). The number of anchor samples per class is set to 2000. Optimizers are Adam optimizers for both the classifier and the anchor distribution with learning rates set to 0.0001. By default, we set the number of iterations of the outer optimization loop to 1. Hence, we have used 30 epochs of training for the classifier using bag loss. Then, we

¹<https://github.com/liujiabin008/LLP-GAN/tree/master>

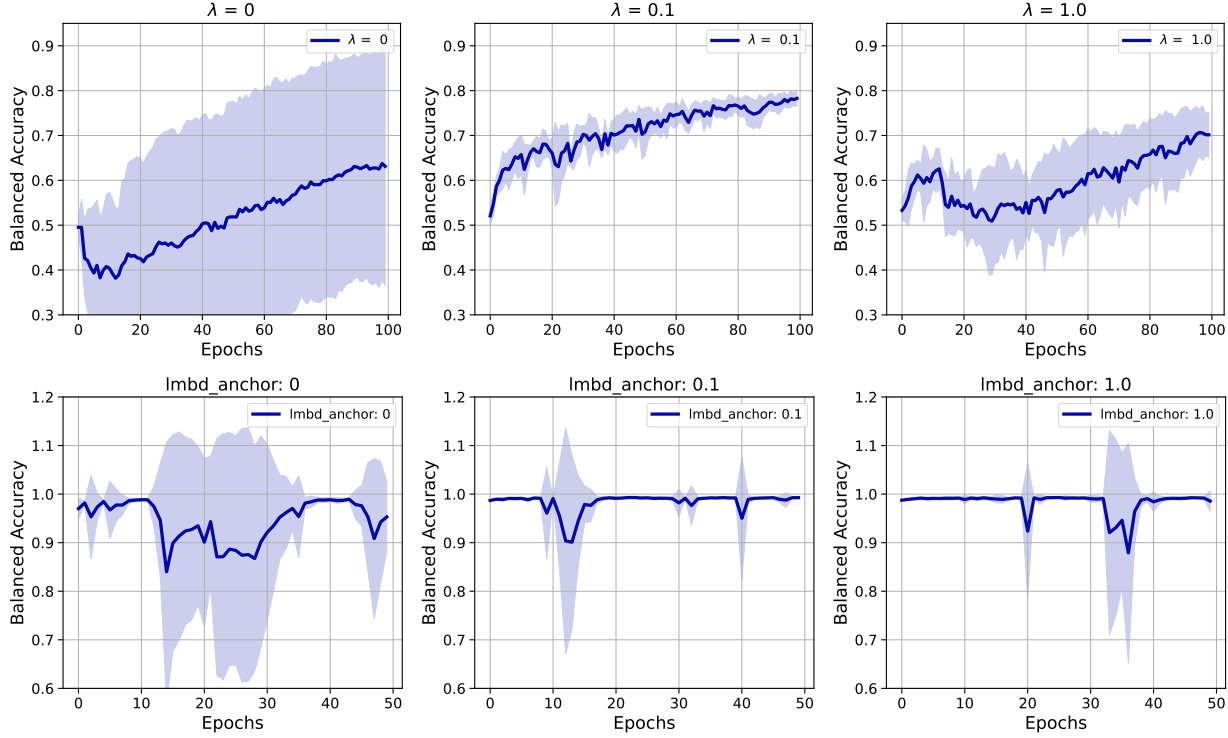


Figure 5. Example of end-to-end learning on CIFAR10 and MNIST for different values of the regularization parameter λ . We can see the benefit of using the anchor-based instance loss (i.e. $\lambda > 0$) either for better performances or for reducing the variance of the results. (top) CIFAR10 results with bags of size 50. (bottom) MNIST results with bags of size 100. We can note that MNIST is an easier dataset and the anchor-based instance loss helps mostly in reducing the variance of the results across the runs.

update the anchor distribution using 3000 iterations and finally we fine-tune the classifier for another 30 epochs using bag loss and instance-level loss on anchor samples. Anchor samples are batched with 8 samples per mini-batch. The regularization parameter λ_B is set to 1 while λ , with $\lambda_W = \lambda$, is validated among $[0.00001, 0.0001, 0.001, 0.01, 0.1]$ optimizer with a learning rate of 0.001 and a batch size of 128.

A.5.4. LIBRARIES

The code has been built using the `Pytorch` library (BSD3) (Paszke et al., 2019) and the `POT` library (MIT) (Flamary et al., 2021) for optimal transport.

A.6. End-to-end experiments on image datasets

We run here some experiments showing that our model can be used in an end-to-end learning fashion and more interestingly it can leverage on some existing pre-trained models. The key adaptations needed to use `FlowLLP` in an end-to-end learning fashion are:

- we use a pre-trained model as a feature extractor to define a latent space. In our experiments, we used ResNet18 pre-trained on ImageNet for CIFAR10, and keep all the layers except the final classification layer. This provides a latent space of dimension 512.
- depending on the latent space dimensionality, we can consider a one-layer neural network, with bias and activation function to reduce the dimensionality of the latent space before feeding it to the anchor distribution learning. In our experiments, we used this projector to obtain a latent space of dimension 50. This projector is particularly useful when the latent space dimensionality is high as Wasserstein distance suffers from the curse of dimensionality.
- anchors are defined then learned in this latent space.
- the classifier, which is a fully connected layer with activation, takes as input embeddings from the pre-trained model (and the dimensionality reduction layer if any) and outputs class probabilities.

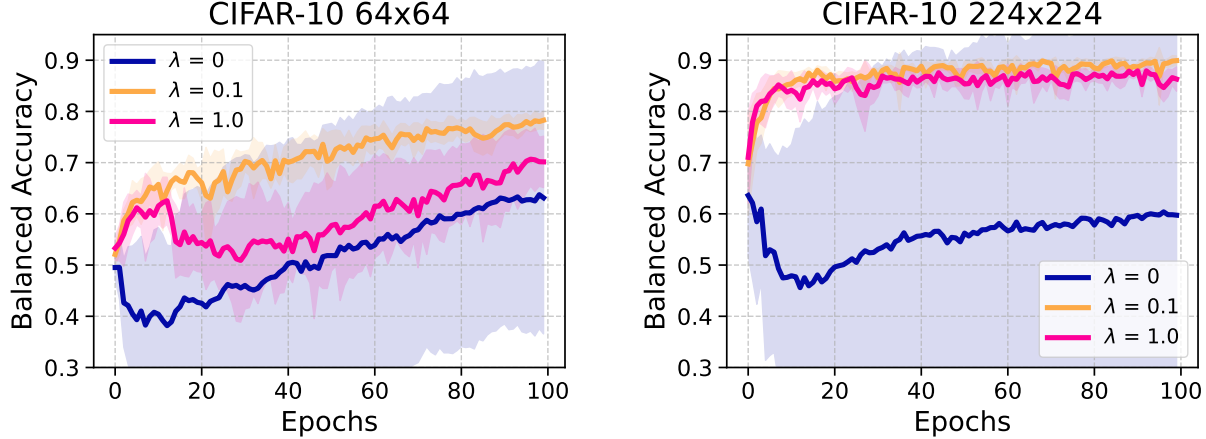


Figure 6. End-to-end training on CIFAR10 with bags of size 50. The anchor space is of dimension 50. We show the evolution of accuracy across epochs during the fine-tuning of f_θ once anchors have been learned. Performance is shown for different values of λ (weight of the anchor loss). (left) image size 64×64 pixels using ResNet18 pre-trained on ImageNet as feature extractor. (right) image size 224×224 pixels.

Based on these adaptations, the anchors live in the latent space defined by the pre-trained model (and the dimensionality reduction layer if any). As such, the anchor loss only impacts the classifier. Instead, the bag-level loss impacts all parts of the model (pre-trained model, dimensionality reduction layer if any, and classifier) as the class prediction of the samples in the bags depend on all these parts. To learn the full model, the training procedure is as follows. At initialisation, we freeze the feature extractor model and train only the dimensionality reduction layer and the classification head using a standard cross-entropy loss on label proportion. After a certain number of epochs, we freeze all the part model and train only the anchor distribution using the particle flow optimization. This happens in the latent space defined by the pre-trained model (and the dimensionality reduction layer if any). Once the anchor distribution is learned, we fine-tuned all models except the anchor distribution using the bag loss and the anchor-based cross-entropy loss as defined in in Equation (3). Parameters for training are : For CIFAR10, images are resized in 64×64 pixels or 224×224 pixels depending on the experiment. We used bags of size 50. and we used 1000 anchors per class, a learning rate of 0.001 for the anchor flow, with $\alpha = 0$ and $T_W = 3000$. For the fine-tuning, we use Adam optimizer with a learning rate of 0.0001, a batch size of 64 and we train for 50 epochs. For the regularization parameter, we used $\lambda_B = 1$ and evaluated the model for different values of λ .

Examples of results on CIFAR10 are reported in Figure 5 and Figure 6. These results are averaged over 10 runs and show the evolution of the accuracy on the test set with respect to the regularization parameter λ and the epochs. We can see that the model can indeed learn in an end-to-end fashion and reach good performances (up to 78% of accuracy on CIFAR10 with bags of size 50). We can also see that anchor-based instance loss is key to reach good performances as without it (i.e. $\lambda = 0$), performances are significantly lower (around 60% of accuracy) and with strong variance across runs. With a proper value of λ (e.g. $\lambda = 0.1$), we can reach good performances with a reduced variance across runs. These results have been obtained with a dimensionality reduction layer to reduce the latent space dimensionality to 50. An ablation study on the impact of this dimensionality reduction layer, detailed in the main paper, have shown that using the original latent space of dimension 512 leads to significantly better performances, and that the higher dimensionality of the latent space does not prevent the model to learn good anchors and reach good performances. We also provide results in Figure 7 showing the impact of multiple iterations of the block coordinate descent. We can see that most of the gain is obtained during the first iteration and that multiple iterations do not improve significantly the performances.

Remark 2. Note that we have also tried to reproduced this end-to-end learning experiment using LLPGAN but failed to reach reasonable performances. The codebase of LLPGAN is publicly available but written in Tensorflow 1.x and thus quite old. We have tried to adapt it to Pytorch with success for the tabular datasets. We believe that the failure to reproduce the end-to-end learning experiment is due to the difficulty to adapt the GAN training to LLP setting. Indeed, LLPGAN uses a more complex discriminator loss and the generator loss is based on an ℓ_2 feature matching loss in the latent space of the discriminator. This makes the training more difficult to stabilize. In addition, the GAN training is known to be quite sensitive to hyperparameters and architecture choices. Overall, these difficulties show us the benefit of using simpler models such as FlowLLP for LLP, that can leverage existing pre-trained models and be trained in an end-to-end fashion with relative ease.

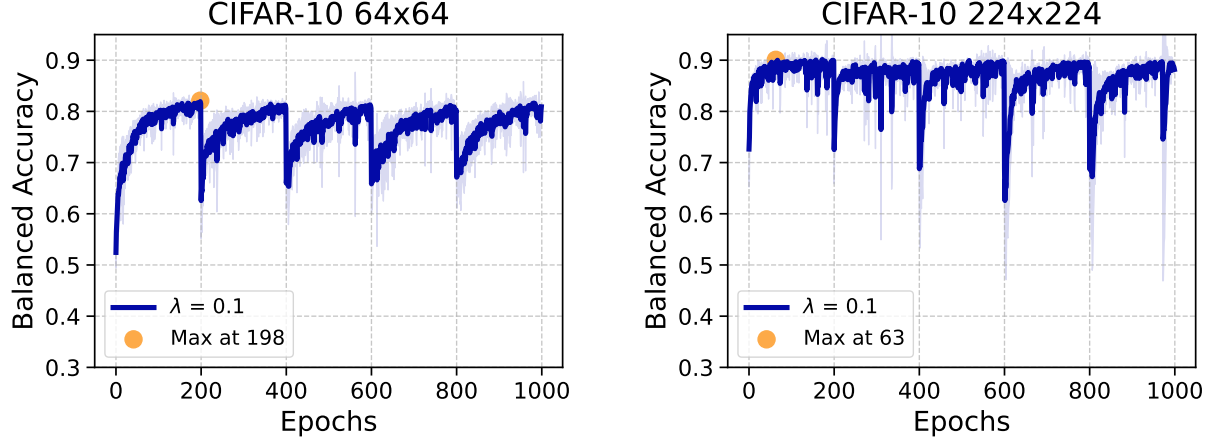


Figure 7. Evolution of the accuracy on CIFAR10 with bags of size 50 on multiple iterations $T = 5$ of the block coordinate descent. (left) image size 64×64 pixels using ResNet18 pre-trained on ImageNet as feature extractor. (right) image size 224×224 pixels. The fine-tuning is done for 200 epochs and then the anchor distribution is updated. This is repeated 5 times. We can see that multiple iterations of the block coordinate descent does not improve significantly the performances and that most of the gain is obtained during the first iteration and the maximum validation accuracy is reached on the first iteration of the block coordinate descent.

A.7. Additional figures for ablations studies of FLOWLLP.

We provide additional ablation studies than those provided in the main part of the paper. For these experiments, bag size is set to 100 samples and all the classes are present in a bag. Other learning settings has been set as in the main performance experiments. The setting of each ablation is described in the caption of each figure. Here is a summary of the different ablation studies provided:

- Impact of the different weights of the two flows (Wasserstein flow and instance-level classifier flow) on the trajectories of the anchors. See Figures 8 to 10.
- Study whether noisy label proportions impact the performance in Figure 11.
- Impact of the number of anchor per class in the anchor distribution in Figure 12.
- Impact of the Gaussian initialization of the anchor distribution in Figure 13.
- Impact of the label regularization in the Wasserstein ground cost in Figure 14.
- Impact of gap and diversity in the label proportion of the bags in Figure 15.
- Analysis of FLOWLLP’s running time with respect to bag size in Figure 16.

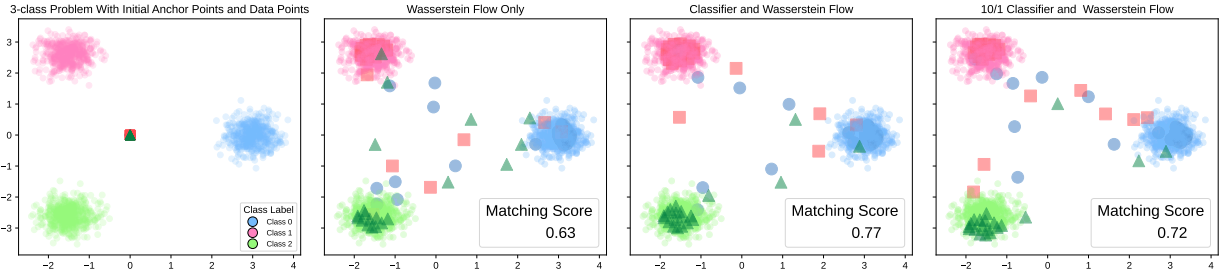


Figure 8. Trajectories of each class-conditional anchors for a 3-class Gaussian problem with 100 bags. The left plots shows class-conditionals of the data distribution as well as the 20 anchors per class. The middle-left plot shows the trajectories of the anchors when using only the Wasserstein flow while the middle-right plot uses both instance-level classifier and Wasserstein flow with equal weights. For the rightmost plot, we have the two flows but with a higher weight on the instance-level classifier. For this experiment, the instance-level initialisation classifier f_θ accuracy is 71% on the bag samples. We quantitatively evaluate the alignment between learned anchors and the empirical data distribution by modeling each class as a multivariate Normal distribution, calculating the log-likelihood of each anchor and reports the percentage of "well-placed" anchors that exceed a log-density threshold of -10 . We note that using equally both Wasserstein and classifier flows yields the best alignment. Using only the Wasserstein flow and over-focusing on the classifier flow lead to sub-optimal alignments.

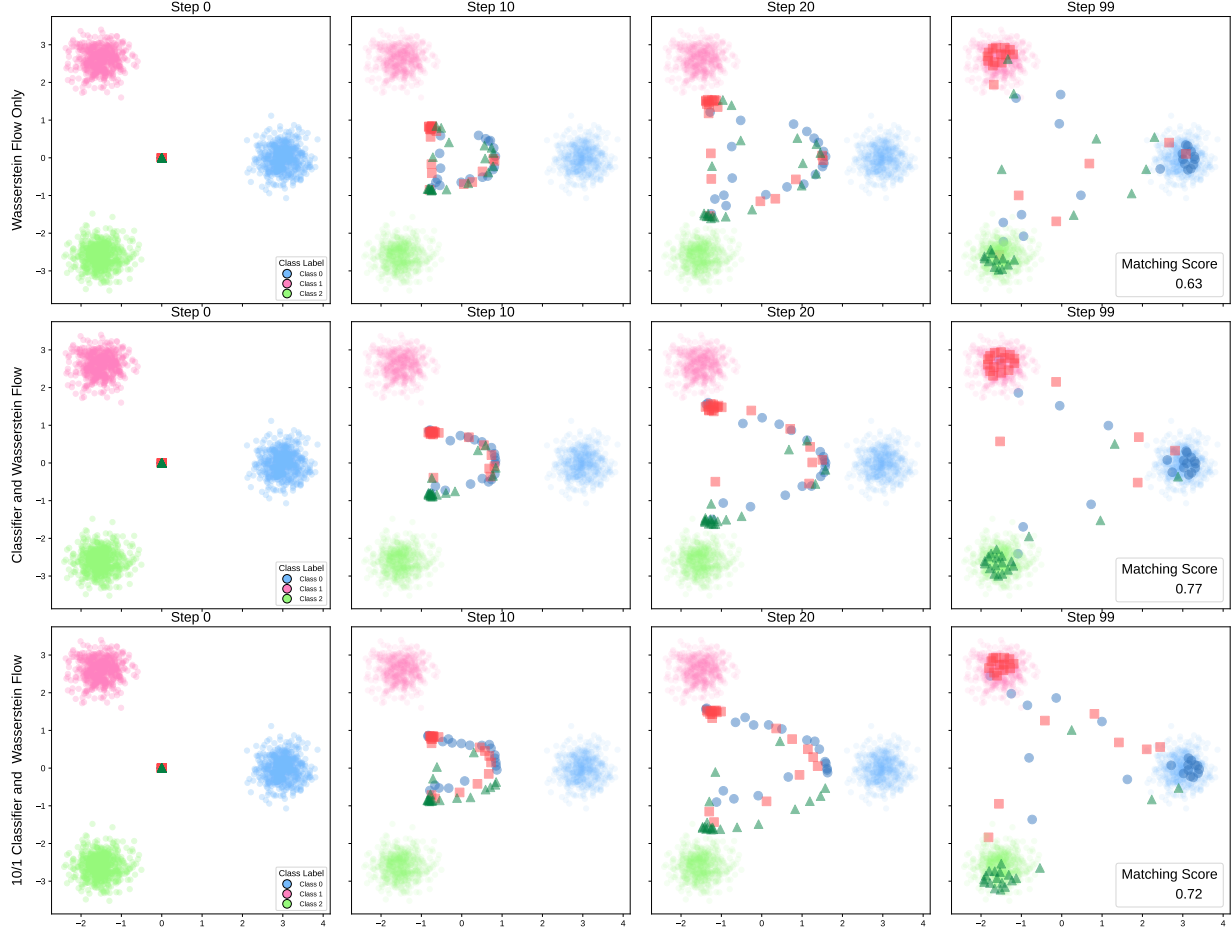


Figure 9. Trajectories of each class-conditional anchors for a 3-class Gaussian problem with 100 bags. 4 steps of the trajectories are shown in columns : from left to right, the initial positions of the anchor and the anchors after 1000, 2000 and 9900 iterations are shown. On each panel, we shows class-conditionals of the data distribution as well as the 20 anchors per class. Each row corresponds to a different setting of weights of the two flow, Wasserstein and instance-level classifier flows. (top) Wasserstein flow only. (middle) Equal weights for both flows. (bottom) Higher weight on the instance-level classifier flow. We quantitatively evaluate the alignment between learned anchors and the empirical data distribution by modeling each class as a multivariate Normal distribution, calculating the log-likelihood of each anchor and reports the percentage of “correctly-located” anchors that exceed a log-density threshold of -10 . In this figure, the accuracy of the initialisation classifier used for the classifier flow is around 71%. We note that using equally both Wasserstein and classifier flows yields the best alignment. Using only the Wasserstein flow and over-focusing on the classifier flow lead to sub-optimal alignments.

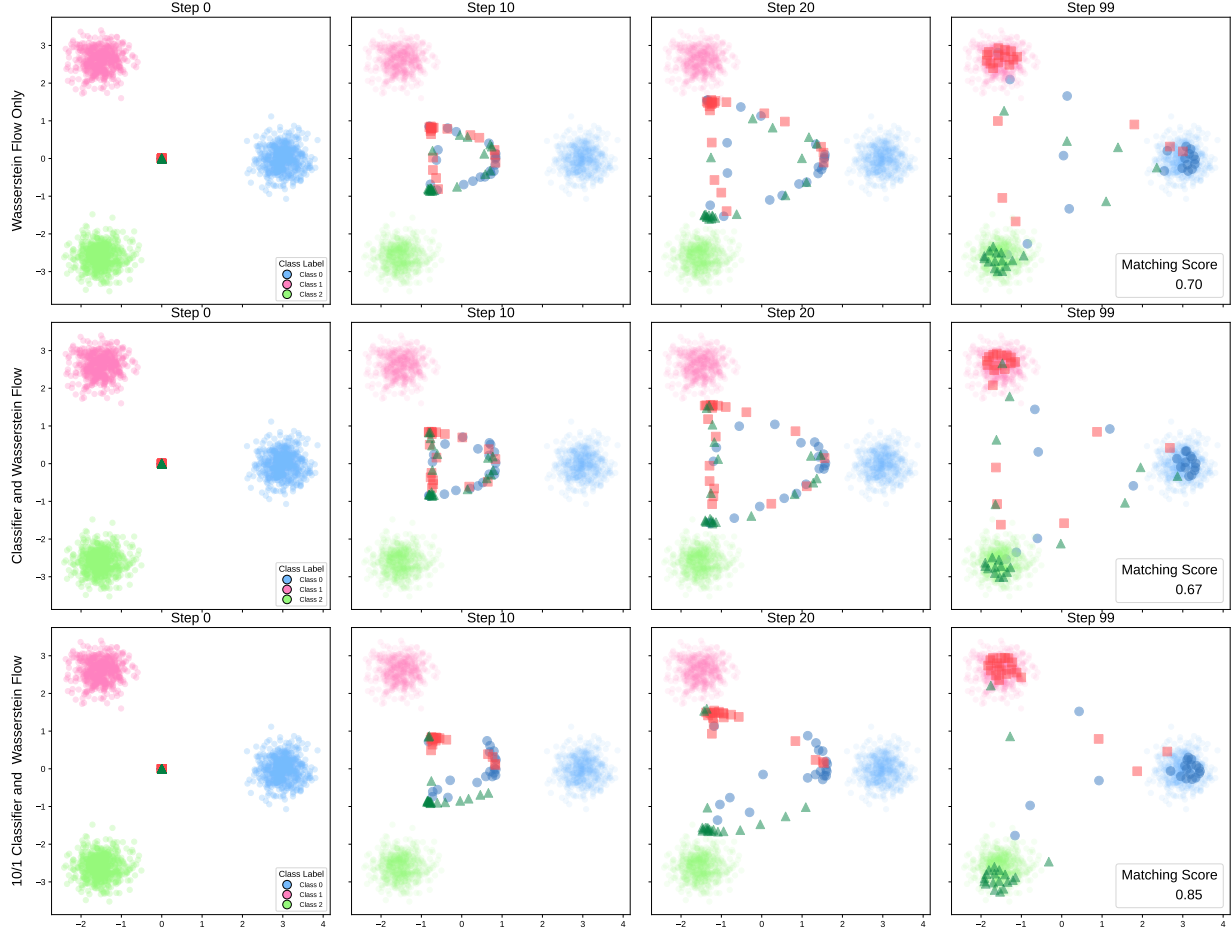


Figure 10. We have the same setting as in Figure 9. In this figure, the accuracy of the initialisation classifier used for the classifier flow is around 92%. Because of this high accuracy, putting more weights on the classifier flow leads to more efficiency at pushing anchors toward their correct class region.

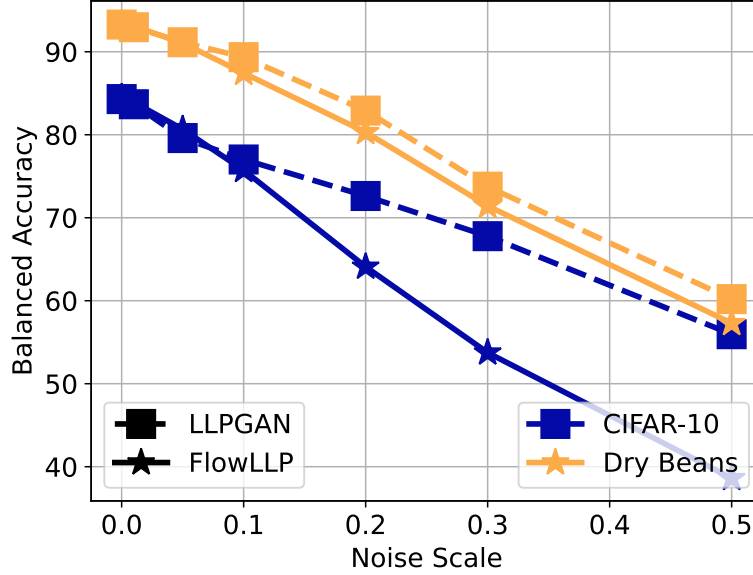


Figure 11. Analysis of the impact of noise on label proportion. Akin to differential privacy, we have evaluated the trade-off between the performance of the algorithms and noise in label proportion. We have considered the same protocols as in the “performance” experiments, but before training the LLP algorithms, we have added a Laplacian noise to each label proportion in the bags. Each proportion is then clipped between 0 and 1 and renormalized to ensure that the sum of the label proportion is 1. We see that when noise increases, performance of all algorithms naturally degrades. However, we can see that `FlowLLP` is more sensitive to noise than `LLPGAN`, especially for large noise levels (greater than 0.1) and depending on the dataset, loss of performance can be important. We explain this by the fact that label proportion noise directly impacts the anchor flow mechanism that is at the core of `FlowLLP`. Indeed, label proportions are used to compute the optimal transport plan that matches anchor samples to samples in the bags. When those proportions are noisy, the matching is less accurate which in turn degrades the performance of the algorithm. Instead `LLPGAN` does not consider label proportions in their GAN mechanism and is thus less sensitive to noise. We see that when noise increases, performance of all algorithms naturally degrades. However, we can see that `FlowLLP` is more sensitive to noise than `LLPGAN`, especially when noise scale is larger than 0.1, and depending on the dataset, loss of performance can be important. We explain this by the fact that label proportion noise directly impacts the anchor flow mechanism that is at the core of `FlowLLP`. Indeed, label proportions are used to compute the optimal transport plan that matches anchor samples to samples in the bags. When those proportions are noisy, the matching is less accurate which in turn degrades the performance of the algorithm. Instead `LLPGAN` does not consider label proportions in their GAN mechanism and is thus less sensitive to noise.

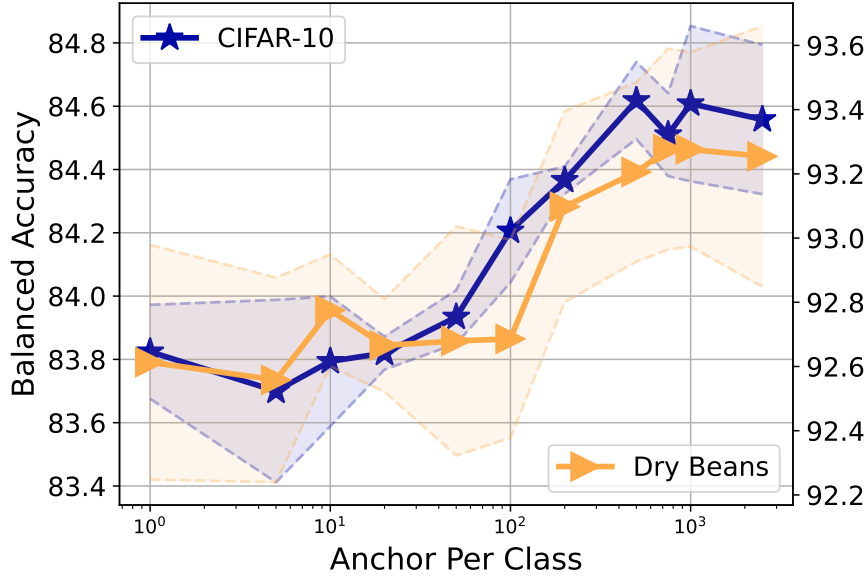


Figure 12. Impact of the number of anchor per class in the anchor distribution. We have considered two multiclass datasets. For this experiment, we have not validate λ and set it to $\lambda = 0.1$ while $\lambda_B = 1$. We note that performances tend to slightly increase with the number of anchor samples. Impacts are dependent on the dataset but start to plateau around 590 anchors per class.

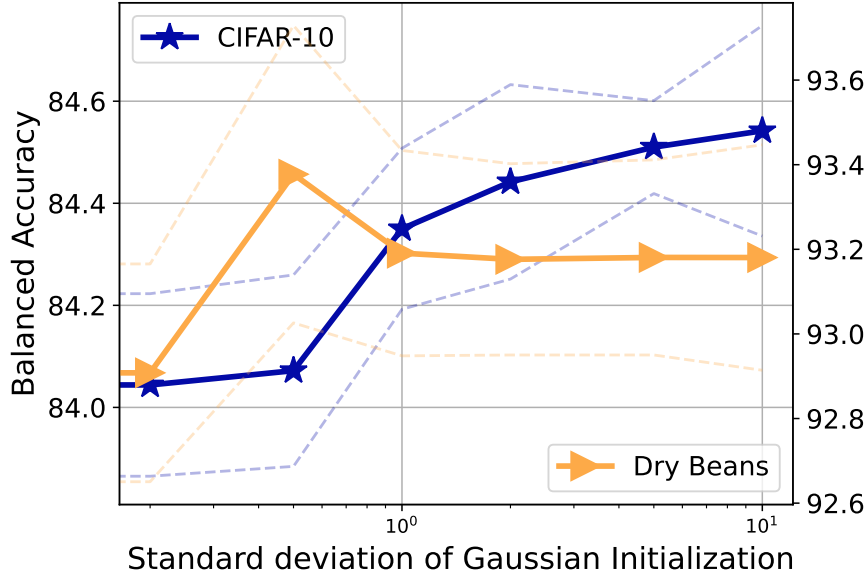


Figure 13. Impact of the initialization of the anchor distribution. Each class-conditional anchor distribution is initialized as a Gaussian distribution with zero-mean and isotropic covariance $\sigma^2 I$. We varied σ from 0.1 to 10. For this experiment, we have not validated λ and set it to $\lambda = 0.1$ while $\lambda_B = 1$. The bag size is 100. We note that performances are quite stable with respect to the initialization of the anchor distribution. For CIFAR, we can see a slight increase of performance between $\sigma = 0$ and $\sigma = 10$ while impacts are negligible on “Dry Beans”. This weak sensitivity to initialization is an important property of the proposed method that makes it robust and easy to use in practice. This robustness is likely due to the classifier flow which pushes anchor samples to flow the right location (region labeled as their class) even if initialization is not ideal.

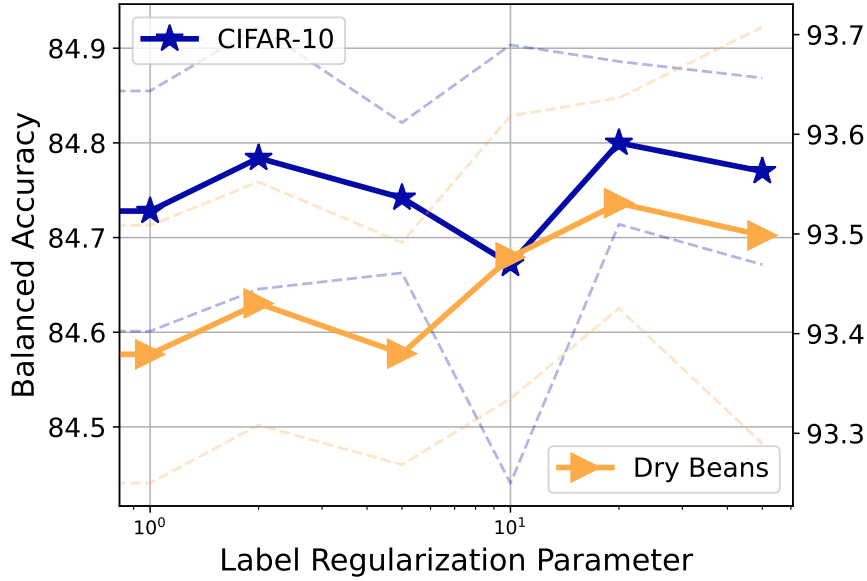


Figure 14. Impact of the regularization parameter α weighting the distance between labels in the ground cost of the OT problem. For this experiment, we have not validated λ and set it to $\lambda = 0.1$ while $\lambda_B = 1$. The bag size is 100. Performances are quite stable with respect to the strength of the label regularization. Variance of the results seems to suggest, that this parameter has a limited impact on the performances. We justify this again by the fact that the classifier flow already takes labels into account when pushing anchor samples to their respective class region.

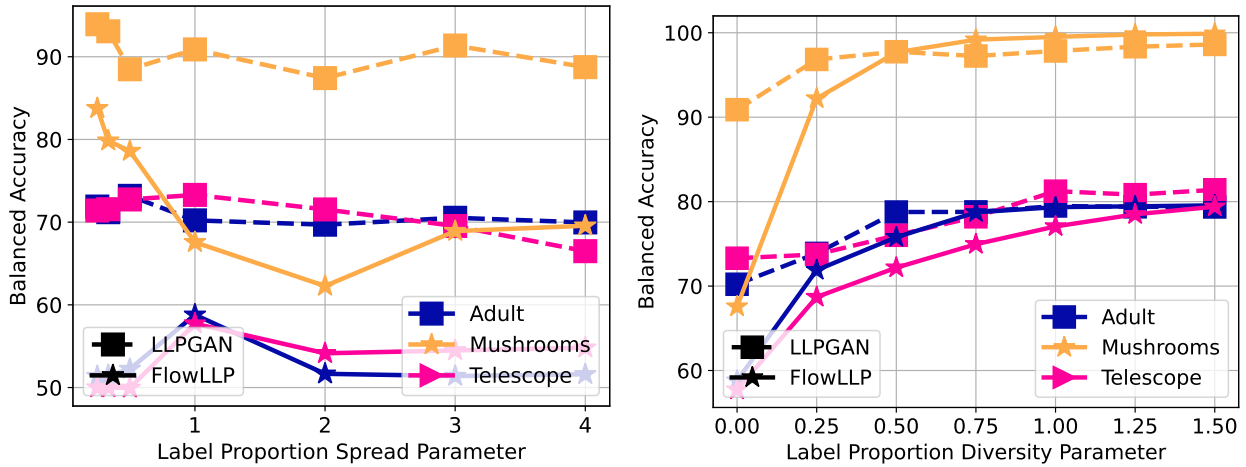


Figure 15. Impact of the label proportion gap and diversity parameters, respectively β and u on performances of two models `FlowLLP` and `LLPGAN`. (left) We varied β from 0.25 to 4 and fixed u to 0. (right) We varied δ from 0 to 1.5, impacting u and fixed β to 1. We note that the gap in label proportion has different impacts depending on the model (see left panel). For `LLPGAN`, increasing the gap tends to (slightly) decrease the performances. For `FlowLLP`, we have a strong loss in performance compared to `LLPGAN`, but the impact of the gap is less clear, with variation, across the datasets and gap width. One conclusion here is that large gap is not sufficient to ensure good performances for `FlowLLP`. Regarding diversity (see right panel), both `FlowLLP` and `LLPGAN` strongly benefit from (strong) diversity in the label proportion. We can note that `FlowLLP` seems to be more sensitive to diversity than `LLPGAN`, which makes sense due to the way the anchor distribution is learned. The important lesson from these experiments is that having diverse bags in terms of label proportion is crucial for good performances and if there is no diversity in label proportions then, `LLPGAN` seems to be more robust than `FlowLLP`.

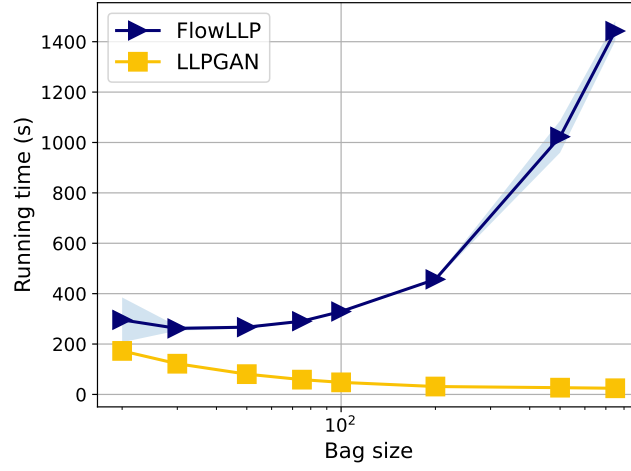


Figure 16. Running time of `FlowLLP` and `LLPGAN` on the 'Indoor' dataset. The training set is split in bags of increasing size. For `FlowLLP`, the running time includes the time for the initialization of the classifier using only bag loss, and one iteration of the anchor particle flow optimization and classifier fine-tuning. Number of epochs for `LLPGAN` has been set to 200 while we use the configuration described in the performance experiment for `FlowLLP`. We can see that `FlowLLP` is slower than `LLPGAN` for all bag sizes. For these tabular datasets, `LLPGAN` is indeed quite fast to train as the GAN mechanism is quite simple (fully connected networks for generator and discriminator), while `FlowLLP` requires to solve an optimal transport problem at each iteration of the anchor flow optimization. Running time of `LLPGAN` decreases with respect to the bag size (the algorithm picks one sample for each bag at each iteration), while `FlowLLP` running time increases with the bag size. This is due to the fact that the number of samples involved in the Wasserstein distance computation increases with the bag size. Note that for the bag size of interest (50 and 100 samples), the running time of `FlowLLP` is higher but still reasonable compared to the one of `LLPGAN`. We want to stress that `FlowLLP` has not been optimized for speed and that using more efficient OT solvers could significantly reduce the running time of `FlowLLP`. Furthermore, we conjecture that for end-to-end learning on images, `FlowLLP` could be faster than `LLPGAN` as the GAN training could be more difficult to stabilize and would require more epochs to converge.